

XERULITH

Manuel de l'utilisateur

Pilotage et exploration autonome de robots et de drones par IA locale — version 0.6.2

EZ-Robot (JD-Humanoid, hexapodes, rovers) · ArduPilot / MAVLink · SunFounder GalaxyRVR · cartes maison · drone, rover et robot humanoïde IA dans Arma 3

by F1GBD (c) 2026 — projet de R&D personnel

Octobre 2026

Sommaire

1. Présentation....4
1.1 Plateformes pilotées....4
1.2 Fonctions communes....4
1.3 L'IA Ollama : un pilotage totalement autonome, en local ou dans le cloud....5
1.4 Arma 3 : formation et test des missions....5
1.5 Ce qui est inclus....6
2. Sécurité....7
2.1 Robot bipède JD....7
2.2 Véhicules ArduPilot, rovers et cartes maison....7
2.3 Arma 3....8
3. Installation....9
3.1 Programme d'installation (recommandé)....9
3.2 Archive autonome....9
3.3 Configuration requise....9
4. Premier démarrage....10
4.1 Connecter une carte EZ-B (JD, hexapode, rover…)....10
4.2 Parcours conseillé....11
4.3 Sans matériel : le simulateur intégré....11
5. L'interface de pilotage....12
5.1 Barre de connexion et pastilles d'état....12
5.2 Caméra....13
5.3 Actions....13
5.4 Déplacement....13
5.5 Servos....14
5.6 Voix....14
5.7 Yeux et pinces....14
5.8 Capteurs et journal....14
6. Profils de robot....15
6.1 Choisir un profil....15
6.2 Profils fournis....16
6.3 L'éditeur de profils....17
6.4 Contrôleurs et déplacements....17
6.5 Capteurs et sorties....18
6.6 Véhicule ArduPilot....19
6.7 SunFounder GalaxyRVR....19
6.8 Drone Arma 3....19
7. L'IA en dialogue (onglet IA — Ollama)....20
8. Exploration autonome....21
8.1 Principe....21
8.2 Modèles....21
8.3 Missions....21
8.4 Modes et garde-fous....22
8.5 Directives du pilote....23
8.6 Position et carte....23
8.7 Journal et rapport de mission....24
9. Drone et rover virtuels dans Arma 3....25
9.1 Principe....25
9.2 Prérequis....26
9.3 Installer les mods....26

9.4 Paramétrer le lanceur d'Arma 3 26
9.5 Préparer la mission 27
9.6 Connecter XERULITH 28
9.7 Piloter le drone 28
9.8 Modes de vol 29
9.9 Réglages du profil drone 30
9.10 Missions autonomes du drone 30
9.11 Rover UGV : exploration terrestre 34
9.12 Formation et sécurité du pont 36
9.13 Dépannage Arma 3 36
10. Robot humanoïde IA dans Arma 3 38
10.1 Préparer le robot 38
10.2 Recherche de victimes (zone contaminée ou hostile) 39
10.3 Reconnaissance et exploration de zone 39
10.4 Mémoire de terrain, curiosité et progrès 40
10.5 Exploration et découverte libre 41
10.6 SATER au sol : balise ELT à l'antenne Yagi 41
11. Monde réel : exploration et découverte libre 43
11.1 Préparer le terrain 43
11.2 Étalonner les pas 43
11.3 Lancer la mission 44
11.4 Ce que fait XERULITH 44
11.5 Rapport et mémoire du lieu 44
12. Scripts Python 45
13. Configurer Ollama 46
13.1 Ollama Cloud (clé API) 46
14. Dépannage 47
15. Données et fichiers 48
16. Crédits 49

1. Présentation

XERULITH est une plateforme de recherche et développement en intelligence artificielle pour le **pilotage et l'exploration autonome de robots et de drones**. Un même poste de pilotage commande des plateformes très différentes, chacune décrite par un **profil**, et peut confier leur pilotage à un grand modèle de langage servi **en local** par Ollama : perception par la caméra et les capteurs, décision par le modèle, action bornée par des garde-fous.

Le projet est né sur un robot humanoïde EZ-Robot JD ; il pilote aujourd'hui des robots EZ-Robot, des véhicules ArduPilot, le rover SunFounder GalaxyRVR, des cartes électroniques maison, et un drone virtuel dans le simulateur **Arma 3**, qui sert à la **formation** et au **test des missions** avant le terrain.

1.1 Plateformes pilotées

Plateforme	Exemples	Ce que XERULITH apporte
EZ-Robot (carte EZ-B v4, Wi-Fi)	JD-Humanoid, hexapodes et quadrupèdes (actions ARC), rovers à servos continus ou à pont en H, tourelles de caméra	Équivalent d'ARC sans restriction : caméra, actions, servos, voix, scripts, IA ; lecture directe des projets ARC
ArduPilot / MAVLink	ArduRover, ArduSub, bateau	Station sol légère : télémétrie (GPS, cap, batterie, télémètres), armement, consignes limitées dans le temps, exploration avec position GPS
SunFounder GalaxyRVR	Rover martien à six roues, firmware d'origine	Caméra, ultrason et infrarouges sans reprogrammation du rover
Cartes maison	ESP32, Arduino en série ou TCP	Commandes texte paramétrables, mesures `CLE=valeur`, GPS NMEA, caméra IP ou USB
Arma 3 (simulation)	Drone de reconnaissance AR-2 Darter	Drone et rover UGV, vol et conduite physiques du jeu, vidéo avec le HUD de XERULITH, télémétrie, missions de formation évaluées
Simulateur intégré	JD simulé, rover virtuel	Essais de l'interface, d'un profil, d'un script ou d'une mission sans matériel

1.2 Fonctions communes

- **Pilotage à distance** : caméra et analyses d'image (couleur, QR, visages, ArUco, mouvement, objets), déplacements au pavé ou au clavier, servos, tête ou tourelle, synthèse et commandes vocales, STOP général.
- **Profils de robot** : un éditeur décrit le contrôleur, le mode de déplacement, les servos, la tête et les capteurs (caméra, télémètre, GPS, cap, profondeur, détecteurs d'obstacle…) ; l'interface et l'IA s'adaptent au profil.
- **IA en dialogue** : on parle au robot en langage naturel ; le modèle propose des ordres, vérifiés avant exécution.
- **Exploration autonome** : une boucle perception, décision, action pilotée par un modèle Ollama (en local, sur le réseau ou dans le cloud), avec une bibliothèque de missions, des garde-fous, un journal et un rapport de mission.
- **Scripts Python** : un programme complet commande le robot actif par l'objet `robot`, quelle que soit la plateforme.

1.3 L'IA Ollama : un pilotage totalement autonome, en local ou dans le cloud

Le cœur de XERULITH est la boucle de **pilotage autonome par IA**. La mission est donnée en français, comme à un équipier : « quadrille la zone, signale tout départ de feu avec sa position ». Ensuite, sans opérateur, le système enchaîne les cycles **perception → décision → action** jusqu'à la fin de la mission :

- un **modèle de vision** (par exemple qwen3-vl) décrit ce que voit la caméra ; les détecteurs locaux (objets, visages, ArUco, points chauds) et tous les capteurs du profil (télémètre, GPS, cap, radiogoniomètre…) complètent l'observation ;
- un **modèle de décision** (par exemple gemma3) raisonne sur la mission, l'observation, sa mémoire et l'historique, puis choisit les déplacements, les notes et les comptes rendus ; il rédige enfin le **rapport de mission** ;
- XERULITH vérifie chaque ordre (noms, bornes, budget, obstacle, zone autorisée) avant de l'exécuter : les **garde-fous restent locaux**, quel que soit le modèle utilisé.

Mode Ollama	Intérêt
Local (PC ou serveur du réseau)	Autonomie complète **sans Internet** : fonctionne sur un réseau isolé ou en situation dégradée ; images et données restent sur place ; latence maîtrisée. Un GPU de 16 Go garde les modèles de vision et de décision chargés ensemble
Cloud (ollama.com, clé API)	Accès à de grands modèles (par exemple gemma4:31b) **sans GPU local**, depuis un simple portable ; même interface, même mission, mêmes garde-fous
Automatique	Réseau local d'abord, cloud en secours si aucun serveur ne répond : la mission continue

Le modèle se choisit dans l'interface, sans rien reprogrammer : on peut comparer plusieurs modèles sur une même mission (de préférence dans Arma 3, § 1.4) et garder le meilleur. Le même pilote IA commande indifféremment un JD, un rover, un véhicule ArduPilot ou le drone simulé : seule la description du profil change.

1.4 Arma 3 : formation et test des missions

La simulation dans Arma 3 est la valeur ajoutée de XERULITH pour la préparation opérationnelle :

- **Former sans risque ni matériel** : opérateurs et pilotes IA s'entraînent sur un drone au comportement réaliste (inertie, vent, relief, carburant), sur des cartes de plusieurs centaines de km², de jour comme de nuit.
- **Tester une mission avant le terrain** : même bibliothèque de missions, mêmes modes (Confirmer, Autonome, Observer), mêmes garde-fous, même journal et même rapport que sur un robot réel. On met au point le texte de la mission, les limites et le choix du modèle Ollama en simulation, puis on passe au matériel.
- **Drone et rover** : un drone AR-2 Darter pour l'exploration aérienne, un rover terrestre **UGV Stomper** pour l'exploration au sol (pente, obstacles, chemin choisi par l'IA conductrice d'Arma), comme un rover d'exploration planétaire.
- **Robot humanoïde IA** : un robot à pied (modèle 3D XERULITH, sans arme) pour la **recherche de victimes** en zone contaminée ou hostile, la **reconnaissance de zone** avec mémoire du terrain, l'**exploration et découverte libre** et le **SATER au sol** à l'antenne Yagi (chapitre 10).
- **Exercices générés au hasard et évalués** : balise de détresse 121,5 MHz à retrouver (SATER), départ de feu à détecter en caméra thermique, reconnaissance de zone, objets à trouver et identifier au sol par le rover, victimes à retrouver et balise ELT à localiser à pied par le robot humanoïde ; le rapport ajoute une **évaluation instructeur** calculée sur la vérité terrain.
- **Comparer et rejouer** : un même scénario sert à comparer des modèles d'IA, des textes de mission ou des réglages, avec des résultats mesurés.

> Parcours conseillé pour toute nouvelle mission : **simulateur intégré** (interface et profil), puis **Arma 3** (déroulé complet de la mission), puis **robot réel** en mode Confirmer chaque plan. Les directives de navigation par points (`ALLER`, `VOLER`, `ROULER`, `HAUTEUR`, `FLIR`) sont propres au drone, au rover et au robot humanoïde Arma 3 ; `SUIVANT` et `OBSERVER` servent aussi au robot réel en exploration et découverte libre (chapitre 11) ; les autres directives sont communes à toutes les plateformes.

1.5 Ce qui est inclus

Élément	Rôle
XERULITH.exe	Le poste de pilotage (Windows 64 bits, sans installation de Python)
Profils de robot	17 profils fournis (JD, hexapode, quadrupède, rover à roues, ArduRover, ArduSub, bateau, GalaxyRVR, drone ArduCopter réel et simulé, rover ArduPilot simulé et par pont LoRa, drone, rover UGV et robot humanoïde Arma 3, carte maison, rover simulé) et leur éditeur
Missions	33 missions d'exploration types, dont 3 pour le drone, 1 pour le rover et 4 pour le robot humanoïde Arma 3, 4 pour ArduPilot et 1 d'exploration et découverte libre pour un robot réel, modifiables et extensibles
Bibliothèques d'actions	39 mouvements d'usine du JD ; lecture des actions des projets ARC (hexapodes, quadrupèdes)
Simulateur	JD simulé (carte EZ-B virtuelle) et rover virtuel pour tout autre profil
Robot humanoïde IA	Mod `@XerulithRobot` (modèle 3D) et mission `mission_robot`, dans `arma3` du dossier d'installation
Repères ArUco	Planche `XERULITH_Reperes_ArUco_10cm.pdf` à imprimer, pour l'exploration du robot réel (chapitre 11)
Pont Arma 3	Mod `@XerulithBridge` (Pythia) et scripts de mission, dans `arma3` du dossier d'installation

> Tout fonctionne en local : aucun service Internet n'est nécessaire (sauf l'option Ollama Cloud).

2. Sécurité

Ces règles sont communes à toutes les plateformes et **toujours actives** :

Règle	Effet
STOP (bouton rouge ou touche **Échap**)	Arrête mouvement, script, exploration et son ; moteurs au neutre, servos **tenus** (jamais relâchés)
Limites du profil	Position de servo hors course **refusée**, pas rognée ; batterie minimale ; distance d'arrêt devant un obstacle
Mouvements limités dans le temps	Chaque pas a une durée ; aucune consigne n'est laissée active ; une liaison perdue arrête le robot
IA bornée	Ordres vérifiés (noms, bornes, budget de pas), acrobaties interdites en exploration, mode **Confirmer** par défaut

2.1 Robot bipède JD

Le JD n'a ni capteur de position sur ses servos, ni capteur de chute, ni capteur de distance. Il a donc des règles propres :

Règle	Pourquoi
STOP (bouton rouge ou touche **Échap**) arrête au prochain point sûr et **tient** la position	Un JD dont les servos sont relâchés s'effondre. Une marche termine son pas avant de s'arrêter, pour rester stable.
Calibrer avant tout mouvement : les 16 servos vont à 90°	Sans retour de position, c'est le seul moyen de connaître la pose. Un mouvement interpolé depuis une pose inconnue est brutal.
Position hors course **refusée**, pas rognée	Chaque articulation a sa course autorisée (tête, pinces, chevilles…).
Batterie : plus aucun mouvement sous **7,20 V**, plus de marche ni d'acrobatie sous **7,45 V**	Une LiPo 2S trop basse fait chuter la tension sous charge et le robot tombe.
Acrobaties au sol seulement avec la case **Acrobaties** cochée	Pompes, poirier, salto… mettent le robot à terre.
Relâcher demande une confirmation explicite	Le robot tombe s'il n'est pas tenu.

> L'exploration autonome d'un robot terrestre se fait **au sol**, dans une zone dégagée, **jamais sur une table** : le robot ne voit pas le vide. Restez à portée du robot pendant toute l'exploration.

2.2 Véhicules ArduPilot, rovers et cartes maison

- Véhicule ArduPilot à **armer** explicitement ; consignes RC limitées à la durée d'un pas, puis rendues à la radiocommande ; failsafe, geofence et contrôles pré-armement d'ArduPilot restent prioritaires.
- Premiers essais **roues levées** ou hélices démontées, radiocommande en main, personne à proximité.
- Rovers et cartes maison : STOP remet les moteurs au neutre ; le GalaxyRVR s'arrête seul après 3 s sans liaison.

2.3 Arma 3

La simulation ne présente aucun risque matériel : c'est l'endroit pour essayer une nouvelle mission en mode Autonome. Le pont n'écoute que sur le PC local et n'accepte qu'un petit nombre d'ordres bornés ; BattlEye doit être désactivé, donc en solo ou sur un serveur local seulement.

3. Installation

3.1 Programme d'installation (recommandé)

1. Téléchargez `XERULITH-0.4.0-setup.exe` depuis la page GitHub du projet.
2. Lancez-le et acceptez la **licence GNU GPL v3** (XERULITH est un logiciel libre ; texte complet installé dans `LICENSE.txt`).
3. L'installation se fait pour l'utilisateur courant, **sans droits administrateur**, dans `%LOCALAPPDATA%\Programs\XERULITH`.
4. Le menu Démarrer reçoit les raccourcis **XERULITH**, **XERULITH (simulateur)**, Lisez-moi, Données XERULITH et la désinstallation. Le raccourci sur le Bureau est en option.

Les mises à jour s'installent par-dessus la version précédente. La désinstallation et les mises à jour conservent le dossier `xerulith_data` : réglages, missions, explorations et journaux.

L'installeur arrête lui-même XERULITH s'il tourne encore. Si un ancien XERULITH est resté bloqué en mémoire (fenêtre fermée mais processus présent), ses fichiers sont mis de côté (`*.xold`, supprimés à l'installation suivante) : **aucun redémarrage du PC n'est nécessaire**. Si un fichier ne peut être ni supprimé ni renommé, l'installeur l'indique : débranchez les périphériques USB ou Bluetooth utilisés (casque, micro, adaptateur série) et relancez-le. Depuis cette version, XERULITH se termine toujours dès la fermeture de sa fenêtre, même si un appel à Ollama est en cours.

3.2 Archive autonome

`XERULITH-v0.4.0-win64.zip` se décompresse n'importe où (clé USB, `C:\XERULITH`…). Lancez `XERULITH.exe` : il doit toujours rester à côté de son dossier `_internal`.

3.3 Configuration requise

Élément	Requis
Système	Windows 10 ou 11, 64 bits
Plateforme	Au choix : robot EZ-Robot (carte EZ-B v4, par exemple JD-Humanoid), véhicule ArduPilot, SunFounder GalaxyRVR, carte maison ; ou aucune (simulateur intégré, Arma 3)
Liaison	Wi-Fi ou réseau local, port série ou radio de télémétrie, selon la plateforme
IA (facultatif)	Serveur Ollama sur le réseau ou sur le PC ; 16 Go de mémoire graphique conseillés pour garder gemma3:12b et qwen3-vl:8b chargés ensemble
Reconnaissance vocale (facultatif)	Micro du PC
Simulation de vol (facultatif)	Arma 3 (Steam) et le mod Pythia, sur le même PC (chapitre 9)

4. Premier démarrage

Choisissez d'abord le **profil** de votre plateforme dans la liste **Robot** de la barre de connexion (chapitre 6), puis connectez-vous :

Plateforme	Adresse à saisir	Détails
Robot EZ-Robot (JD, hexapode, rover EZ-B…)	IP de la carte, par exemple `192.168.1.65` (bouton **Scanner**)	§ 4.1
Véhicule ArduPilot	`udp:0.0.0.0:14550`, `COM7`, `tcp:192.168.2.2:5760`…	§ 6.6
SunFounder GalaxyRVR	`192.168.4.1` (Wi-Fi du rover)	§ 6.7
Carte maison	`192.168.4.1:8023` (TCP) ou `COM5` (série)	§ 6.4 et 6.5
Drone Arma 3	`127.0.0.1:47800` (pont Arma 3)	chapitre 9
Aucune	case **Simulateur**	§ 4.3

4.1 Connecter une carte EZ-B (JD, hexapode, rover…)

1. Allumez le robot (un JD **debout**, ou tenu). Attendez que la carte EZ-B soit prête.
2. **Fermez ARC** : la carte EZ-B n'accepte qu'un seul client à la fois.
3. Le réseau dépend du mode de la carte. En mode **point d'accès**, connectez le PC au Wi-Fi du robot : son adresse est `192.168.1.1`. En mode **client**, le robot est sur votre box, avec l'adresse que celle-ci lui donne.
4. Cliquez 🔍 **Scanner** : XERULITH balaye le réseau du PC et liste les cartes trouvées. Double-cliquez sur la vôtre, ou saisissez l'adresse à la main (`192.168.1.65:23`, le port est facultatif).
5. Cliquez **Connecter**. Les pastilles en bas à droite passent à « CONNECTÉ » et « ROBOT », et la batterie s'affiche.
6. Si le profil exige un calibrage (JD, hexapode…), cliquez **Calibrer** et confirmez : les servos rejoignent le centre de leur course d'un coup. La pastille « CALIBRÉ » s'allume.



Recherche de la carte EZ-B sur le réseau (bouton Scanner).

Le Scanner envoie l'octet de poignée de main de l'EZ-B (0x55) sur le port 23 de chaque adresse ; une EZ-B v4 répond 0x2A. Une carte déjà tenue par ARC apparaît en orange, avec la mention « port 23 occupé ».

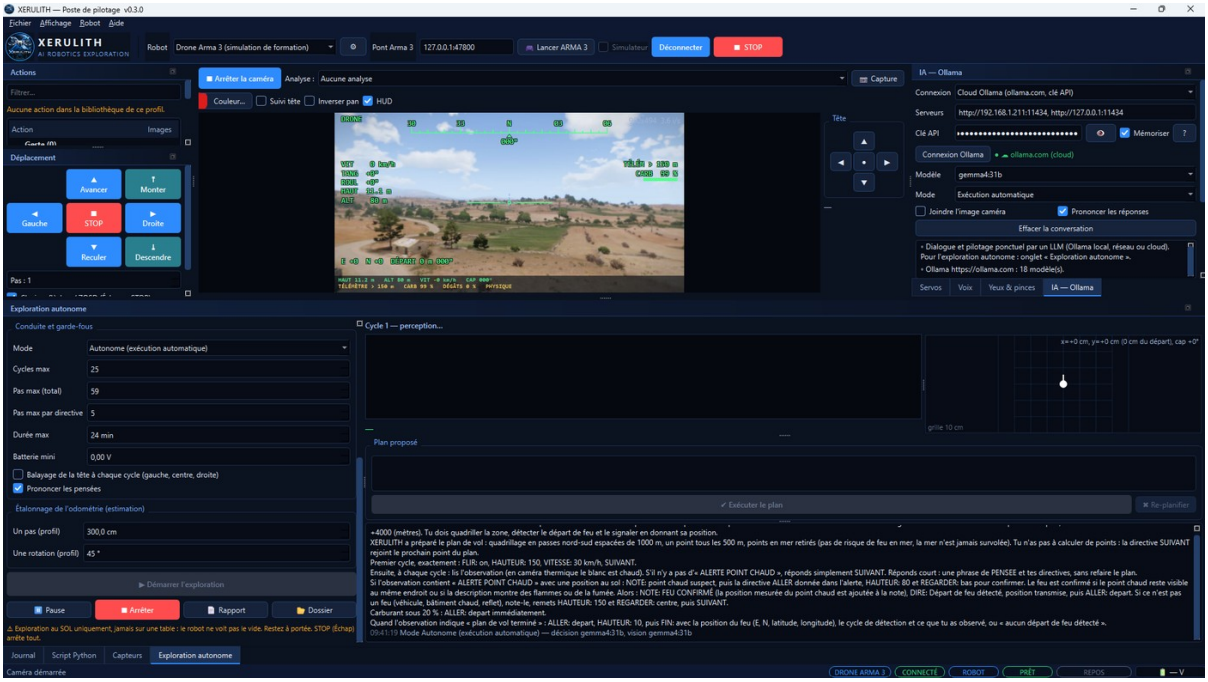
4.2 Parcours conseillé

1. **Simulateur intégré** : prendre en main l'interface, mettre au point un profil, un script ou une mission en mode **Observer**.
2. **Arma 3** : dérouler la mission complète avec un drone virtuel, en mode **Autonome**, et lire le rapport (chapitre 9).
3. **Plateforme réelle** : rejouer la mission en mode **Confirmer chaque plan**, dans une zone dégagée, STOP à portée de main.

4.3 Sans matériel : le simulateur intégré

Cochez **Simulateur** avant de cliquer Connecter, ou lancez le raccourci **XERULITH (simulateur)**. Le profil JD utilise un JD simulé ; tout autre profil, un rover virtuel dans une pièce de 4 m × 3 m, avec télémètre, GPS et cap. Toutes les fonctions sont disponibles : mouvements, caméra de synthèse, batterie simulée (maintenue chargée par défaut, voir l'onglet Capteurs) et exploration autonome.

5. L'interface de pilotage



Le poste de pilotage connecté au drone Arma 3 : caméra avec le HUD de XERULITH, actions, déplacement, IA — Ollama, exploration autonome.

La fenêtre est organisée en panneaux, comme ARC. Chaque panneau se déplace par son titre, s'empile en onglets ou se détache en fenêtre. **Affichage → Disposition par défaut** remet tout en place. Au démarrage, un panneau fermé ou détaché revient automatiquement à sa place.

5.1 Barre de connexion et pastilles d'état

Commande	Effet
Robot / ⚙️	Profil du robot piloté, éditeur de profils (chapitre 6)
Adresse / Scanner / Simulateur	Adresse du robot (IP, port COM, chaîne MAVLink, pont Arma 3), recherche des cartes EZ-B sur le réseau, robot virtuel
Connecter / Déconnecter (**F5**)	La déconnexion laisse les servos en l'état (un JD reste debout)
Calibrer	Servos au centre de leur course, obligatoire après chaque connexion pour les profils qui l'exigent (JD…)
STOP (Échap)	Arrête mouvement, script, exploration et son ; servos tenus
Relâcher…	Relâche tous les servos après confirmation (le robot tombe s'il n'est pas tenu)
Acrobaties	Autorise les actions au sol (confirmation demandée)
Yeux I2C	Active la carte des yeux RGB du JD
Armer / Désarmer	Véhicule ArduPilot seulement (chapitre 6.6)

Raccourci	Effet (partout dans la fenêtre)
Échap	STOP
F5	Connecter / déconnecter le robot du profil
F6	Démarrer la mission choisie dans le panneau Exploration autonome, ou l'arrêter

Raccourci	Effet (partout dans la fenêtre)
Flèches, ZQSD, Page précédente / suivante	Déplacement au clavier (hors champs de saisie)
H	RTH : retour au point de départ (drone ou rover Arma 3, véhicule ArduPilot ; clavier activé dans le panneau Déplacement)
Ctrl+Alt+X (global)	Revenir sur XERULITH depuis n'importe quelle fenêtre, Arma 3 compris : fenêtre au premier plan, souris libérée
Ctrl+Alt+S (global)	STOP du drone, du rover ou du robot sans quitter Arma 3

Disposition enregistrée : à la fermeture, XERULITH enregistre la taille et la position de la fenêtre, les panneaux affichés ou fermés, leur place, leurs onglets et la taille de leurs séparateurs (panneau Exploration, Caméra…), et les rétablit au démarrage. Un panneau fermé reste fermé : le menu **Affichage** le rouvre. **Affichage → Disposition par défaut** remet tout en place.

Retour depuis Arma 3 : tant que sa fenêtre a le focus, Arma 3 retient la souris. **Ctrl+Alt+X** ramène sur XERULITH à tout moment (raccourci global, actif même dans le jeu) et **Ctrl+Alt+S** donne le STOP depuis Arma. XERULITH reprend aussi la main tout seul dès qu'il est connecté au pont, et libère la souris quand sa fenêtre est réactivée (clic ou Alt+Tab). Si un autre programme utilise déjà ces raccourcis, le journal le signale.

En bas à droite, les pastilles indiquent le profil, la connexion, le mode (ROBOT ou SIMULATION), la pose (CALIBRÉ ou POSE ?), l'action en cours et la tension de la batterie. La barre de batterie est verte au-dessus de 7,6 V, orange entre 7,2 et 7,6 V, rouge en dessous.

5.2 Caméra

- **Démarrer la caméra** : flux en direct de la caméra du profil (EZ-B, ESP32-CAM, caméra IP ou USB, capture de la fenêtre d'Arma 3), avec le nombre d'images par seconde affiché.
- **Analyse** : couleur (choix par le bouton Couleur…), QR codes, visages, marqueurs ArUco, mouvement, objets. La détection d'objets demande le module onnxruntime et un modèle à installer à part.
- **Suivi tête** : la tête ou la tourelle suit la cible détectée. Si elle part du mauvais côté, cochez **Inverser pan**.
- **Pavé Tête** : déplacement par petits pas (maintenir pour répéter) ; le bouton central recentre la tête.
- **Capture** : enregistre l'image dans `xerulith_data\captures`.

5.3 Actions

Les actions du profil (les 39 mouvements de la bibliothèque du JD, ou celles d'un projet ARC pour un hexapode ou un quadrupède) sont classées en **Geste**, **Déplacement** et **Au sol (acrobatie)**. Double-cliquez sur un mouvement ou utilisez **Jouer**, en réglant le nombre de boucles et la vitesse. La barre de progression suit les images du mouvement. Les acrobaties ne sont jouées que si la case Acrobaties est cochée.

5.4 Déplacement

Pavé Avancer / Reculer / Gauche / Droite et nombre de pas. Avec l'option clavier, les **flèches** ou **ZQSD** pilotent le robot, et **Échap** l'arrête. La liste des poses (STAND, Calibrate…) applique une pose de la bibliothèque.

⌂ **RTH** (bouton orange en bas à gauche du pavé, touche **H**) : retour au **point de départ initial**. Il est affiché pour le drone et le rover Arma 3 et pour les véhicules ArduPilot. Une mission en cours est d'abord arrêtée, ainsi qu'un mouvement, puis le véhicule rejoint son départ : la position à la connexion, ou au lancement de la dernière mission. Arma 3 : trajet `ALLER: depart` à la hauteur courante (repère **H**

de la bande de cap) ; ArduPilot : passage en mode **RTL** (paramètre `mode_rth` du contrôleur pour un autre mode). STOP interrompt le retour.

5.5 Servos

Un curseur par articulation, avec son nom et sa course autorisée. Un servo ne bouge qu'au relâchement du curseur, et uniquement après Calibrer. **Lire les positions** resynchronise les curseurs ; **Copier la pose (JSON)** place la pose courante dans le presse-papiers.

5.6 Voix

- **Synthèse vocale** : moteur Windows (SAPI, voix françaises installées dans Windows) ou Piper (voix neuronales locales). Le son est joué par le haut-parleur d'une carte EZ-B, sinon par le PC.
- **Phrases rapides**, débit et volume réglables.
- **Reconnaissance vocale** : maintenez le bouton 🎤 et parlez dans le micro du PC. Les phrases reconnues déclenchent les commandes vocales (« avance », « salue », « stop »…), modifiables par **Éditer les commandes…**. Une phrase non reconnue peut être envoyée à l'IA.

5.7 Yeux et pinces

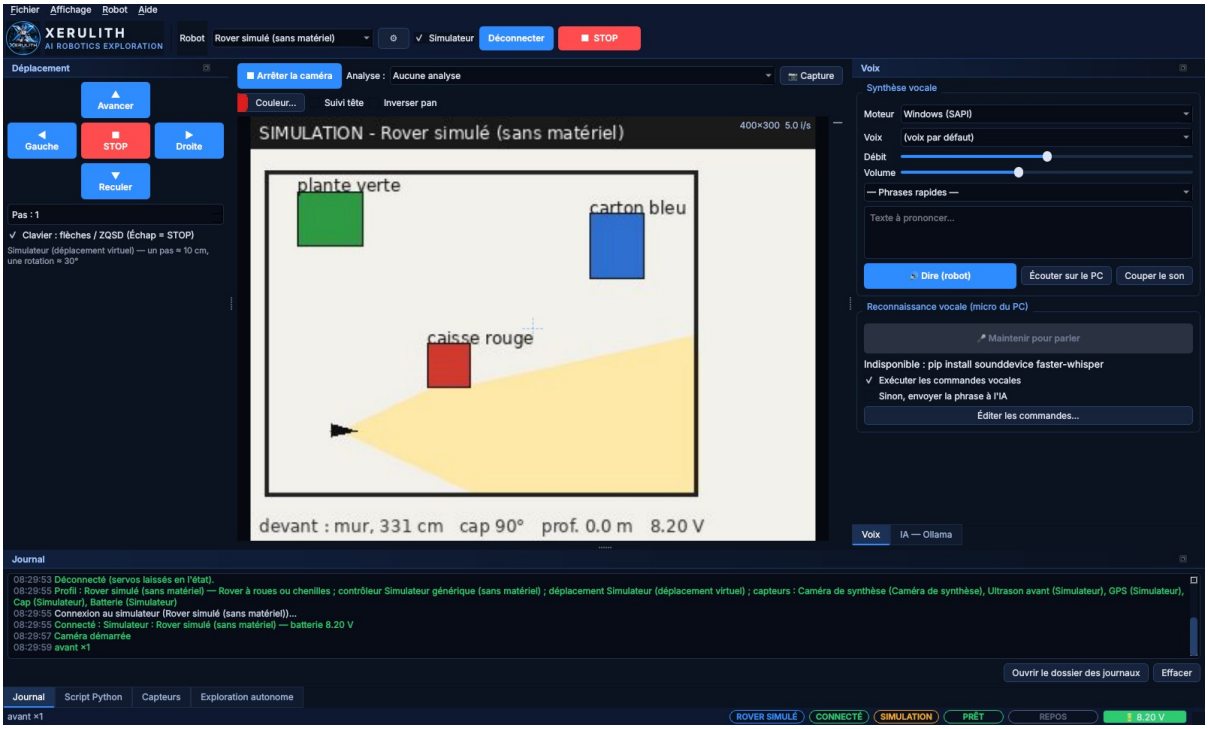
Panneau affiché pour les robots qui en ont (JD). Animations des yeux RGB (13 animations) ou couleur fixe ; ouverture des pinces gauche et droite de 0 à 100 %.

5.8 Capteurs et journal

L'onglet **Capteurs** affiche en direct les mesures de chaque capteur du profil (distance, GPS, cap, profondeur, hauteur, radiogoniomètre…), trace la tension de la batterie (la position GPS s'affiche en degrés, minutes, secondes, par exemple 48°34'48.4"N 003°00'13.4"E, suivie des degrés décimaux) et, pour une carte EZ-B, la température de son processeur et ses entrées analogiques et numériques. Le **Journal** horodate tous les événements ; il est aussi enregistré dans `xerulith_data\logs`.

6. Profils de robot

XERULITH pilote le JD, mais aussi d'autres robots. Un **profil** décrit le robot : son type, son **contrôleur**, sa façon de **se déplacer**, ses **servos** et sa **tête**, ses **capteurs** et la source de chacun, ses fonctions et ses seuils de sécurité. Les profils sont des fichiers JSON rangés dans `xerulith_data\profiles`.



Profil « Rover simulé » : la caméra de synthèse montre la pièce vue de dessus et le champ du télémètre.

6.1 Choisir un profil

La liste **Robot** de la barre de connexion choisit le profil ; le bouton ⚙️ (ou **Robot → Profils de robot…**) ouvre l'éditeur. On ne change de profil que robot **déconnecté**. Chaque profil garde sa propre adresse de connexion. L'interface s'adapte au profil :

Élément	Selon le profil
Adresse	IP de l'EZ-B, IP:port d'une carte maison, port série (COM5…), chaîne de connexion MAVLink ; aucune pour le simulateur
Calibrer	Affiché si le profil exige un calibrage (servos au centre de leur course)
Armer	Affiché pour un véhicule ArduPilot ; la pastille indique ARMÉ ou DÉSARMÉ
Actions, Servos, Yeux et pinces	Masqués si le robot n'a pas de bibliothèque d'actions, de servos, d'yeux ou de pinces
Déplacement	Boutons **Monter** et **Descendre** (touches Page précédente / Page suivante) pour un sous-marin
Capteurs	Tableau des mesures en direct de chaque capteur du profil ; seuil de batterie du profil
IA et exploration	Description du robot, ordres permis, distance d'un pas et seuil de batterie tirés du profil

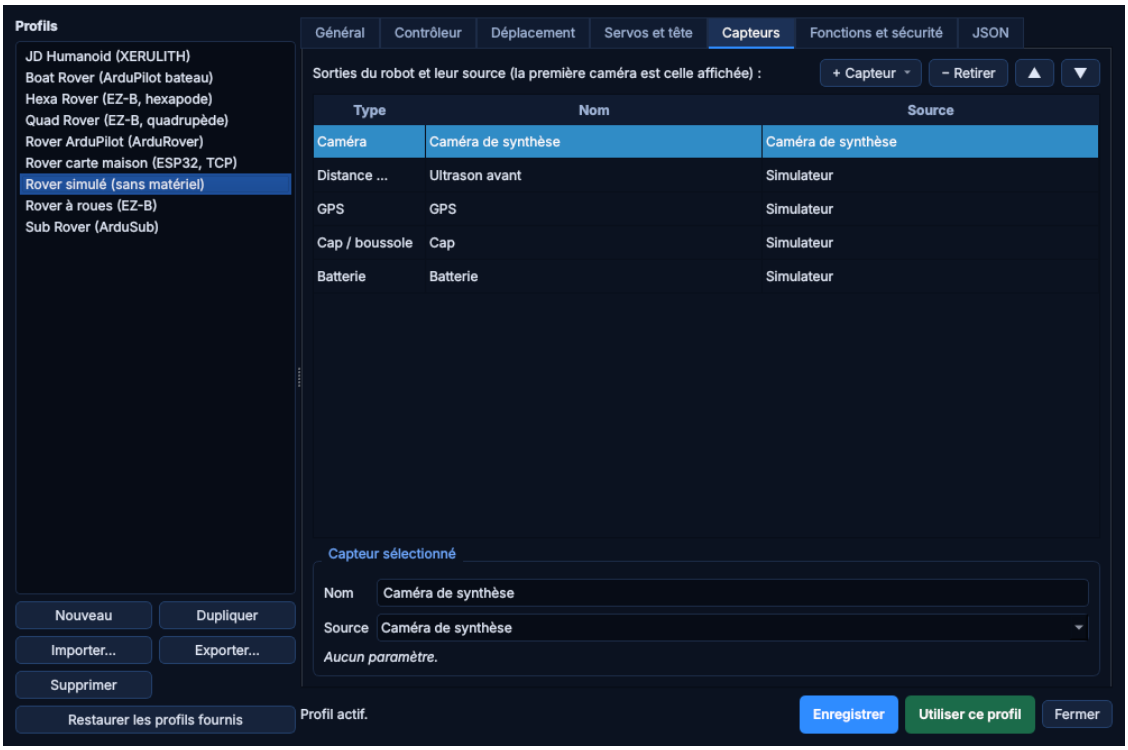
6.2 Profils fournis

Profil	Contrôleur et déplacement	Capteurs
JD Humanoid (XERULITH)	EZ-B v4, actions du JD	Caméra, batterie
Hexa Rover	EZ-B v4, actions d'un projet ARC (Auto Position)	Caméra, ultrason I2C, batterie
Quad Rover	EZ-B v4, actions d'un projet ARC	Caméra, ultrason analogique, batterie
Rover à roues	EZ-B v4, deux servos à rotation continue, tourelle de caméra	Caméra, ultrason, GPS sur UART, batterie
Rover ArduPilot	ArduRover, consignes RC	Caméra IP, GPS, cap, batterie, télémètre
Sub Rover	ArduSub, consignes RC (avance, lacet, vertical)	Caméra IP, profondeur, cap, batterie
Boat Rover	ArduRover cadre bateau	Caméra IP, GPS, cap, batterie
SunFounder GalaxyRVR	Wi-Fi, firmware SunFounder d'origine	Caméra ESP32-CAM, ultrason, 2 infrarouges, batterie
Drone Arma 3 (simulation de formation)	Pont Arma 3 (Pythia), consignes de vol du drone	Capture de la fenêtre du jeu, télémètre, position, cap, hauteur, vitesse, carburant, radiogoniomètre 121,5 MHz
Rover UGV Arma 3 (simulation de formation)	Pont Arma 3, conduite du rover UGV Stomper	Capture de la fenêtre du jeu, télémètre, position, cap, vitesse, carburant, tangage, roulis
Robot humanoïde Arma 3 (simulation de formation)	Pont Arma 3, marche du robot humanoïde IA (chapitre 10)	Caméra de tête, télémètre, position, cap, vitesse, batterie, rayonnement ; antenne Yagi 121,5 MHz en mission SATER
Drone ArduCopter (Pixhawk) / (SITL)	MAVLink, mode GUIDED ; SITL : simulateur ArduPilot hors ligne	GPS, cap, batterie, hauteur, vitesse, télémètre ; caméra de nacelle (RTSP) sur le drone réel
Rover ArduPilot (SITL) / via pont LoRa 868	MAVLink, mode GUIDED ; simulateur ArduPilot ou liaison radio LoRa 868 MHz (Heltec V3)	GPS, cap, batterie, télémètre
Rover carte maison	ESP32 en TCP, commandes texte	ESP32-CAM, ultrason, GPS, batterie
Rover simulé	Simulateur, pièce de 4 m × 3 m	Caméra de synthèse, ultrason, GPS, cap, batterie

Le simulateur ArduPilot, Mission Planner OFF-GRID et le pont LoRa 868 MHz ont chacun leur fiche : **Fiche simulateur ArduPilot / Mission Planner**, **Fiche pont LoRa 868 MHz** et **Fiche rover hors réseau**, dans la documentation de XERULITH.

Ces profils sont des points de départ : dupliquez-les et adaptez ports, canaux et capteurs à votre robot. Un profil fourni supprimé ne revient pas, sauf avec **Restaurer les profils fournis** ; le profil du JD ne peut pas être supprimé.

6.3 L'éditeur de profils



L'éditeur de profils, onglet Capteurs.

À gauche, la liste des profils : **Nouveau**, **Dupliquer**, **Supprimer**, **Importer…** et **Exporter…** (fichiers JSON à partager). À droite, la fiche en onglets :

Onglet	Contenu
Général	Nom, type de robot, description lue par l'IA (une phrase qui dit quel corps elle commande)
Contrôleur	EZ-B v4, ArduPilot / MAVLink, carte maison série ou TCP, simulateur ; paramètres de connexion
Déplacement	Mode de déplacement et ses paramètres ; distance d'un pas et rotation d'un pas (odométrie de l'exploration)
Servos et tête	Servos EZ-B (port, nom, course min et max) ; ports panoramique et inclinaison de la tête, centres, amplitude du balayage d'exploration
Capteurs	Les sorties du robot : type, nom, source et paramètres de chacune ; la première caméra est celle affichée
Fonctions et sécurité	Calibrage obligatoire, actions, yeux, pinces, haut-parleur ; batterie minimale et distance d'arrêt
JSON	Le profil complet, modifiable à la main puis **Appliquer le JSON à la fiche**

Enregistrer valide et sauvegarde le profil ; **Utiliser ce profil** le sauvegarde et le sélectionne. Les incohérences sont refusées : port de servo hors de D0 à D23 ou déclaré deux fois, course invalide, mode de déplacement incompatible avec le contrôleur.

6.4 Contrôleurs et déplacements

Déplacement	Contrôleur	Paramètres
Actions du JD	EZ-B v4	Noms des actions Avancer, Reculer, Gauche, Droite (Forward, Reverse, Left, Right)
Actions d'une	EZ-B v4	Projet ARC (.EZB) contenant un Auto Position, ou

Déplacement	Contrôleur	Paramètres
bibliothèque		fichier .json de Commander ; nom de l'action de chaque direction. Les servos sont commandés dans la course déclarée par le profil
Servos à rotation continue	EZ-B v4	Ports gauche et droit, neutre (90°), amplitude, inversion de chaque roue, durée d'un pas
Pont en H	EZ-B v4	Sorties IN1 à IN4, deux sorties PWM, vitesse en %, durée d'un pas (L298N, TB6612…)
Consignes RC ArduPilot	MAVLink	Canaux d'avance, de rotation et vertical, PWM neutre, amplitude, avance en virage (direction Ackermann), durée d'un pas
Commandes texte	Carte maison	Une commande par direction et une commande d'arrêt ; `{v}` vitesse, `{ms}` durée, `{n}` nombre de pas
Moteurs du GalaxyRVR	GalaxyRVR	Vitesse en ligne droite et en rotation (%), durée d'un pas et d'une rotation
Consignes du drone	Arma 3	Vitesses horizontale et verticale, vitesse de rotation, durée d'un pas, vitesse de transit, rayon et hauteur maximaux (§ 9.9)
Simulateur	Simulateur	Distance d'un pas et rotation d'un pas

Canaux RC usuels : ArduRover **1** direction et **3** gaz ; ArduSub **3** vertical, **4** lacet et **5** avance. Un pas d'une carte maison envoie par exemple `F 60 600`, puis `S` à la fin du mouvement ou sur STOP.

6.5 Capteurs et sorties

Capteur	Sources
Caméra	EZ-B (port 24), flux réseau MJPEG / RTSP / HTTP (ESP32-CAM, caméra IP, téléphone), caméra USB du PC, caméra de synthèse, capture d'écran de la fenêtre d'un simulateur (Arma 3)
Distance	Ultrason I2C SRF02 / SRF08 / SRF10 sur l'EZ-B (adresse 0x70), capteur analogique sur un ADC de l'EZ-B (MaxBotix, Sharp : cm par unité), DISTANCE_SENSOR d'ArduPilot, valeur d'une carte maison
GPS	Trames NMEA sur une UART de l'EZ-B, sur un port série du PC ou en TCP ; GPS d'ArduPilot ; position d'une carte maison
Batterie	Tension mesurée par l'EZ-B, pont diviseur sur un ADC (volts par unité), SYS_STATUS d'ArduPilot, carte maison
Cap, profondeur	ArduPilot (ArduSub pour la profondeur), carte maison
Obstacle	Détecteur infrarouge ou contact : entrée numérique de l'EZ-B (actif à l'état bas), carte maison (1 = obstacle), infrarouges gauche et droit du GalaxyRVR
Entrées	ADC de l'EZ-B (échelle, unité) ; entrée numérique de l'EZ-B (contact, fin de course)
Mesures du drone Arma 3	Télémètre avant (150 m), position, cap, et en entrée analogique : `hauteur`, `altitude`, `vitesse`, `carburant`, `degats`, `balise_cap`, `balise_rel`, `balise_dbm`

Une carte maison renvoie ses mesures par lignes de texte `CLE=valeur` (séparées par des espaces ou des points-virgules), ou par des trames NMEA. La clé de chaque capteur se règle dans le profil :

```
DIST=42 BAT=7.62
GPS=48.5436,3.0036;CAP=270
$GPGGA,123519,4807.038,N,01131.000,E,1,08,0.9,545.4,M,46.9,M,,*47
```

En exploration, les mesures entrent dans l'observation du pilote, et **AVANCER est refusé** si un capteur de distance mesure moins que la **distance d'arrêt** du profil, ou si un détecteur d'obstacle est déclenché.

6.6 Véhicule ArduPilot

1. Reliez la télémétrie : radio USB (`COM7`), Wi-Fi ou MAVProxy (`udp:0.0.0.0:14550`), compagnon réseau (`tcp:192.168.2.2:5760`). La saisie de la barre de connexion remplace la valeur du profil.
2. **Connecter** attend le battement du véhicule (10 s au plus). GPS, cap, batterie et télémètres arrivent dans l'onglet Capteurs.
3. **Armer** passe dans le mode du profil (MANUAL par défaut) puis arme le véhicule, après confirmation. Aucun déplacement n'est accepté tant que le véhicule est désarmé.
4. Chaque pas envoie des consignes RC pendant sa durée, remet les canaux au neutre puis les rend à la radiocommande. STOP coupe le mouvement en cours de la même façon.

> Premiers essais roues levées ou hélices démontées, radiocommande en main pour reprendre la main, aucune personne à proximité. Les sécurités d'ArduPilot (failsafe, geofence, contrôles pré-armement) restent actives et prioritaires.

6.7 SunFounder GalaxyRVR

Le rover martien GalaxyRVR (carte compatible Arduino Uno, ESP32-CAM, 6 roues) se pilote avec son **firmware d'origine** : XERULITH parle à l'ESP32-CAM comme l'application SunFounder Controller, qu'il faut fermer.

1. Connectez le PC au Wi-Fi **GalaxyRVR** (mot de passe `12345678`), ou mettez le rover et le PC sur la même box.
2. Choisissez le profil **SunFounder GalaxyRVR**. L'adresse est `192.168.4.1` en point d'accès, sinon l'IP donnée par la box. L'interrupteur de téléversement de la carte doit être sur la position de fonctionnement.
3. **Connecter** : XERULITH reconnaît le firmware 2.x (port 30102, trames binaires) ou 1.x (port 8765, JSON). Le protocole peut être imposé dans le profil.
4. La caméra lit le flux de l'ESP32-CAM (`http://<ip>:9000/mjpg`) ; le pavé Tête incline la caméra. L'onglet Capteurs montre l'ultrason, les deux infrarouges et la batterie.

Tant que XERULITH est connecté, il renvoie l'état du rover toutes les 0,3 s. Si la liaison Wi-Fi est coupée plus de 3 s, l'ESP32 arrête le rover de lui-même.

6.8 Drone Arma 3

Le profil **Drone Arma 3 (simulation de formation)** pilote un drone AR-2 Darter dans le simulateur Arma 3, par le mod Pythia et le pont `@XerulithBridge` ; le profil **Rover UGV Arma 3 (simulation de formation)** pilote de la même façon un rover terrestre UGV Stomper (§ 9.11), et le profil **Robot humanoïde Arma 3 (simulation de formation)** un robot humanoïde à pied (chapitre 10). L'installation des mods, le paramétrage du lanceur d'Arma, la mission, le pilotage et les missions autonomes sont détaillés au **chapitre 9**.

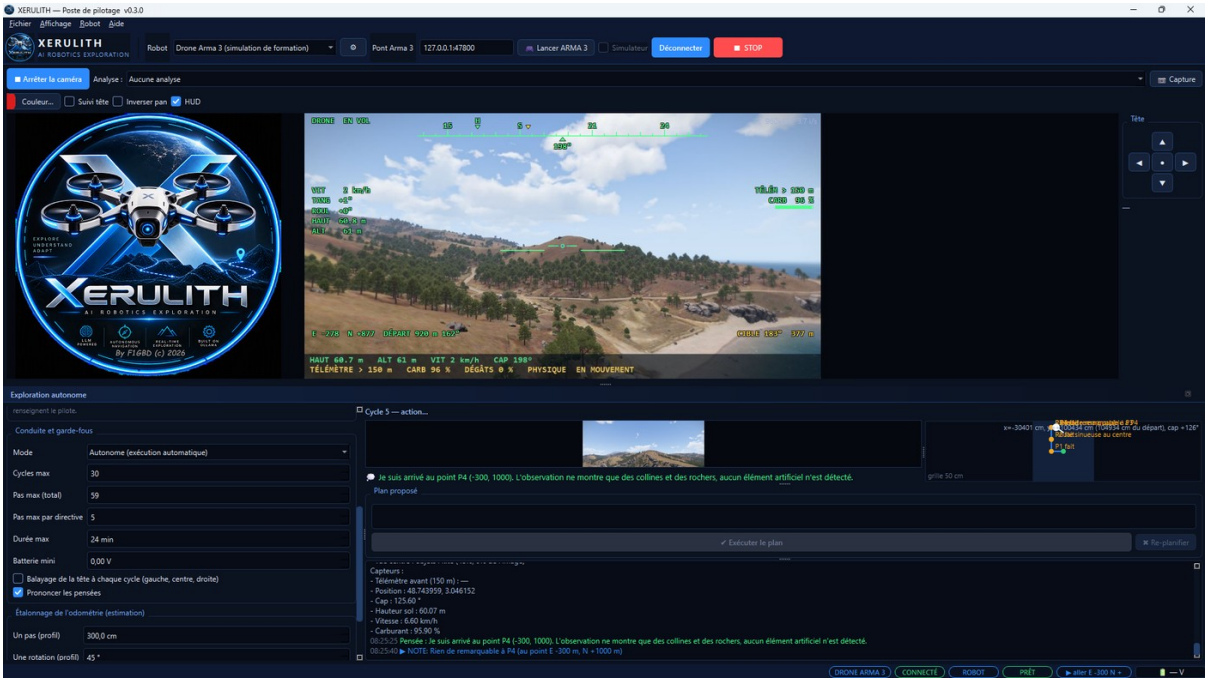
7. L'IA en dialogue (onglet IA — Ollama)

Cet onglet sert au pilotage ponctuel : vous parlez à XERULITH en langage naturel, et le LLM répond en proposant des ordres.

1. Choisissez la **Connexion** : **Réseau local** (serveurs de la liste), **Cloud Ollama** (ollama.com, avec une clé API) ou **Automatique** (réseau, puis cloud si aucun serveur ne répond). Voir le chapitre 13.
2. En réseau, vérifiez les **Serveurs** : par défaut `http://192.168.1.211:11434` (serveur M1A), puis `http://127.0.0.1:11434` (Ollama local). Le premier qui répond est utilisé.
3. Cliquez **Connexion Ollama** : le voyant passe au vert et la liste des modèles se remplit.
4. Choisissez le **Modèle** et le **Mode**. **Confirmer chaque directive** est conseillé : les ordres proposés sont listés et vous cochez ceux à exécuter. En mode **Exécution automatique**, ils sont exécutés directement ; en mode **Conversation seule**, rien n'est exécuté.
5. Écrivez, par exemple : « salue tout le monde puis avance de deux pas ». **Joindre l'image caméra** envoie l'image courante, ce qui demande un modèle vision.

Toutes les sécurités du robot restent actives : un ordre hors course, une acrobatie non autorisée ou un nom d'action inconnu sont signalés et refusés.

8. Exploration autonome



Exploration autonome avec le drone Arma 3 : vidéo et HUD, réglages de la mission, panorama du cycle, pensée du pilote, trajectoire, journal.

8.1 Principe

Chaque **cycle** d'exploration enchaîne quatre étapes :

1. **Perception** — la tête balaie gauche, centre et droite ; les trois images forment un panorama. Les détecteurs locaux (visages, ArUco, objets) l'analysent, et le **modèle vision** le décrit en quelques phrases.
2. **Décision** — le **modèle de décision** reçoit la mission, l'observation, la batterie, la position estimée, sa mémoire (les NOTE) et les derniers cycles. Il répond par une **PENSEE** et 1 à 3 ordres.
3. **Garde-fous** — chaque ordre est vérifié : noms connus, budget de pas, gestes uniquement (jamais d'acrobatie), refus d'avancer face à un obstacle proche détecté.
4. **Action** — selon le mode : validation de l'opérateur, exécution automatique, ou rien (mode Observer).

8.2 Modèles

À chaque démarrage, XERULITH impose **gemma3:12b** pour la décision et **qwen3-vl:8b** pour la vision. Vous pouvez en changer pendant la session ; le bouton **⟳ Modèles** recharge la liste du serveur. Si un modèle manque, le journal donne la commande à lancer sur le serveur Ollama :

```
ollama pull gemma3:12b
ollama pull qwen3-vl:8b
```

Joindre le panorama au modèle de décision envoie aussi l'image au modèle de décision (s'il sait lire les images) en plus de la description du modèle vision.

8.3 Missions

La mission est le texte envoyé au pilote à chaque cycle. La liste déroulante propose la **bibliothèque de missions**, rangée par catégorie :

Catégorie	Exemples
Découverte	Explorer et décrire ; inventaire de la pièce ; objet le plus grand ; tour de la zone ; **exploration et découverte libre (robot réel)** (chapitre 11)
Recherche et sauvetage	Recherche de victime en spirale ; balise de détresse (ArUco 7) ; reconnaissance de zone sinistrée
Navigation par repères	Parcours ArUco 1 à 4 ; aller-retour au marqueur 1 ; recenser les marqueurs
Interaction humaine	Robot d'accueil ; cache-cache ; chercher une personne et la saluer
Surveillance	Gardien de nuit ; recherche d'un objet rouge
Tests de comportement	Contournement d'obstacle ; labyrinthe (main droite) ; règles conditionnelles
Drone Arma 3 (formation)	Reconnaissance 1 km² et retour ; SATER : recherche de balise 121,5 MHz ; surveillance incendie FLIR 20 km² (§ 9.10)
Rover UGV Arma 3 (formation)	Exploration terrestre rover UGV 1 km² (§ 9.11)
Robot humanoïde Arma 3 (formation)	Recherche de victimes après un SATER (zone contaminée et hostile) ; reconnaissance et exploration de zone (mémoire de terrain) ; exploration et découverte libre ; SATER, balise ELT 121,5 MHz à l'antenne Yagi (chapitre 10)
ArduPilot (essai SITL / réel)	Missions d'essai du drone et du rover ArduPilot, au simulateur puis sur le terrain (fiche simulateur ArduPilot)

- 💾 **Enregistrer…** ajoute le texte courant à la bibliothèque (titre et catégorie demandés ; un titre existant est remplacé après confirmation).
- 🗑️ supprime la mission choisie ; ✎ **Fichier** ouvre `missions.json` dans l'éditeur ; ⟳ recharge la liste.

> Une bonne mission donne une **condition de fin** claire (« termine quand… »), **une seule chose à chercher**, nommée simplement, et des comportements attendus explicites (« dis… », « salue… », « note… »). Testez toujours une nouvelle mission en mode **Observer** avant de faire bouger le robot.

Pour les missions ArUco, imprimez des marqueurs du dictionnaire **4x4_50** de 8 à 10 cm de côté : la planche `XERULITH_Reperes_ArUco_10cm.pdf` en donne de 10 cm, prêts à imprimer à l'échelle 100 %.

8.4 Modes et garde-fous

Réglage	Rôle (valeur par défaut)
Mode	Confirmer chaque plan (défaut), Autonome, ou Observer
Cycles max	Nombre maximal de cycles (30)
Pas max (total)	Budget de pas de toute la mission (60)
Pas max par directive	Plafond d'un ordre AVANCER, RECULER ou TOURNER (3)
Durée max	Durée maximale de la mission (20 min)
Batterie mini	Arrêt de la mission sous cette tension (7,5 V)
Balayage de la tête	Panorama gauche, centre, droite à chaque cycle (coché)
Prononcer les pensées	Le robot dit sa pensée à chaque cycle (décoché)

En mode **Confirmer**, le plan proposé s'affiche avec une case par ordre : décochez ce que vous refusez puis cliquez ✔️ **Exécuter le plan**, ou ✖️ **Re-planifier** pour que le pilote propose autre chose. Le mode **Autonome** demande une confirmation au lancement sur le vrai robot.

Pause suspend la mission à la fin du cycle en cours ; **Arrêter**, STOP ou Échap la terminent.

8.5 Directives du pilote

Directive	Effet
`AVANCER: n` / `RECULER: n`	n cycles de marche avant ou arrière
`TOURNER: gauche|droite n`	n rotations sur place
`MONTER: n` / `DESCENDRE: n`	Sous-marin : remonter ou plonger ; drone : monter ou descendre
`ALLER: est nord [vitesse]` / `ALLER: depart`	Drone Arma 3 : rejoindre un point (m depuis le départ), ou revenir au départ (§ 9.10)
`VOLER: cap distance [vitesse]`	Drone Arma 3 : voler la distance au cap vrai
`HAUTEUR: m` / `VITESSE: m/s|max`	Drone Arma 3 : hauteur sol ; vitesse des trajets suivants
`FLIR: on|off`	Drone Arma 3 : caméra thermique (blanc = chaud), points chauds localisés au sol
`ROULER: cap distance [vitesse]`	Rover et robot humanoïde Arma 3 : aller à la distance au cap vrai, par le chemin choisi par l'IA d'Arma
`SUIVANT`	Point suivant du parcours préparé par XERULITH ; en découverte libre, piste d'exploration la plus prometteuse (chapitres 10 et 11)
`OBSERVER: n` / `RAMASSER: n`	Robot humanoïde : s'approcher de l'élément n et l'examiner ; prendre un petit objet en main, le décrire et le reposer. Robot réel : `OBSERVER` seulement
`ENTRER: n` / `EMBARQUER: n` / `QUITTER`	Robot humanoïde Arma 3 : visiter un bâtiment ; conduire un véhicule non armé ; sortir, descendre ou reposer l'objet tenu
`YAGI`	Robot humanoïde Arma 3, SATER au sol : tour d'horizon radio à l'antenne Yagi (§ 10.6)
`REGARDER: gauche|droite|centre|haut|bas`	Oriente la tête
`DIRE: texte`	Phrase prononcée par le haut-parleur
`NOTE: texte`	Mémorise un fait (objet, repère) avec la position estimée
`ACTION: geste`	Geste de la bibliothèque (Wave, Bow, Point…)
`ATTENDRE: s`	Pause en secondes
`FIN: bilan`	Termine la mission

8.6 Position et carte

Le JD n'a ni centrale inertielle ni capteur de distance : la position est une **estimation** par comptage des pas et des rotations. Étalonnez-la dans **Étalonnage de l'odométrie** : mesurez la distance parcourue par un pas « Forward » (3 cm par défaut) et l'angle d'une rotation « Left » ou « Right » (15° par défaut). La carte trace la trajectoire, le cap du robot et les notes, sur une grille de 10 cm. Pour un autre robot, la distance d'un pas et la rotation d'un pas viennent de son profil. Pour le drone, le rover et le robot humanoïde Arma 3, la position n'est pas estimée mais **mesurée** dans le simulateur, en mètres depuis le point de départ. En exploration et découverte libre d'un robot réel, les **repères ArUco** recalent l'estimation (chapitre 11).

Anti-boucle. Si le pilote IA répète la même action sans aucun progrès (pas de déplacement, rien de nouveau vu ni examiné), XERULITH la lui signale, l'interdit pendant 6 cycles et, après 2 cycles sans action utile, relance lui-même la suite de la mission (`SUIVANT` ou `YAGI`).

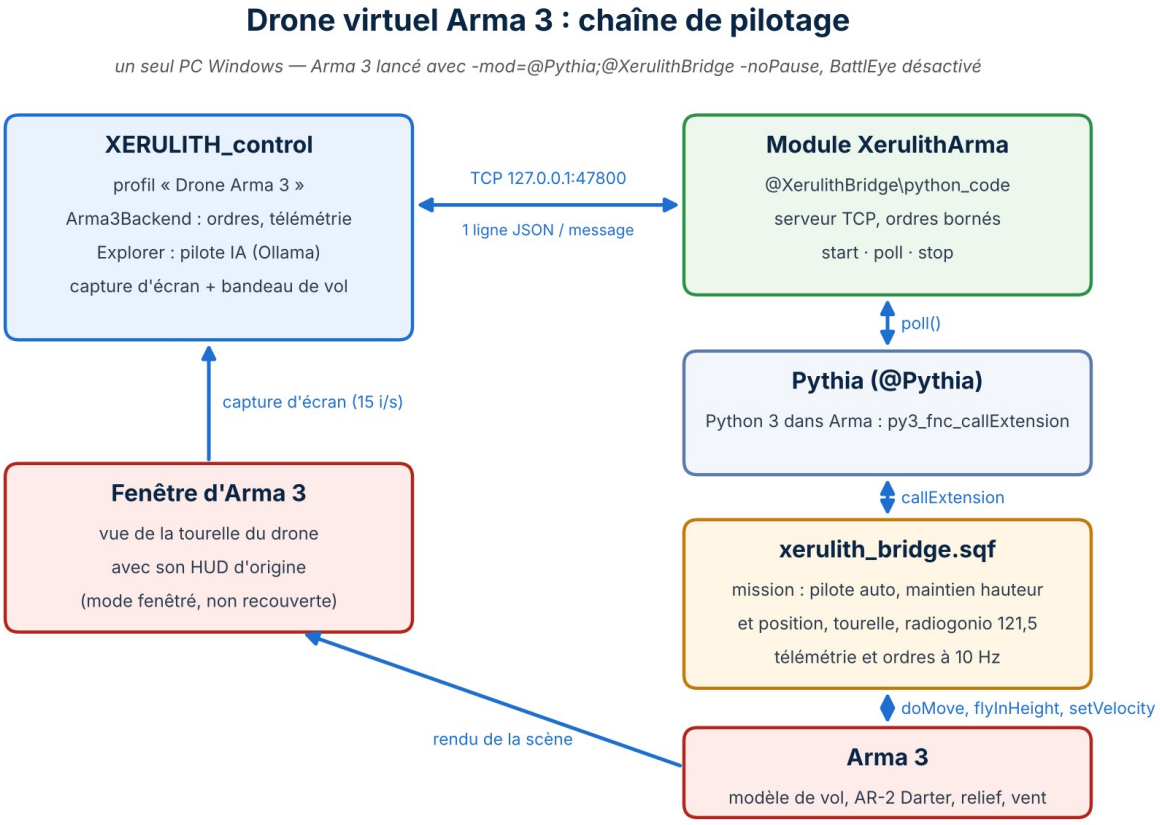
8.7 Journal et rapport de mission

Chaque mission est enregistrée dans `xerulith_data\explorations\<date>\` :

- `config.json` : les réglages de la mission ;
- `journal.jsonl` : pour chaque cycle, l'observation, la réponse du LLM, les ordres et leurs résultats ;
- `cycle_NNN.jpg` : les panoramas ;
- `rapport.md` : le rapport de mission rédigé par le LLM en fin de mission (bouton 📄 **Rapport**).

9. Drone et rover virtuels dans Arma 3

XERULITH pilote un drone de reconnaissance **AR-2 Darter** dans le simulateur **Arma 3**, avec le **modèle de vol du jeu** : inertie, inclinaison en accélération, vent, freinage. Le pilotage se fait au pavé comme pour un robot réel, par script, ou par le **pilote autonome** (Ollama) pour des missions de reconnaissance, de recherche SATER ou de surveillance incendie ; un **rover terrestre UGV** se pilote de la même façon pour l'exploration au sol (§ 9.11). La vidéo est la vue d'une caméra fixée au véhicule, capturée dans la fenêtre du jeu, sans l'interface d'Arma : le **HUD est dessiné par XERULITH** à partir de la télémétrie (§ 9.7) ; la télémétrie (position, cap, hauteur, vitesse, télémètre, radiogoniomètre 121,5 MHz) arrive dans XERULITH dix fois par seconde.



Chaîne de pilotage : XERULITH, le pont @XerulithBridge (Pythia) et le script de mission.

9.1 Principe

Élément	Rôle
Pythia (mod `@Pythia`)	Exécute du Python 3 dans Arma 3 ; le script de mission l'appelle par `py3_fnc_callExtension`
@XerulithBridge	Mod livré avec XERULITH : module Python `XerulithArma` (serveur TCP sur `127.0.0.1:47800`, ordres bornés) et un addon minimal `xerulith_bridge.pbo`
init.sqf	Script d'exemple de la mission : crée le drone devant le joueur, à 10 m de hauteur, et lance le pont
xerulith_bridge.sqf	Script du pont dans la mission : caméra du véhicule, pilote automatique (drone) ou conduite (rover), maintien de hauteur et de position, télémétrie, exercices (balise 121,5 MHz, feu, objets)
XERULITH	Profil « Drone Arma 3 », connexion au pont, capture de la fenêtre du jeu, bandeau de télémétrie, pilote IA

Tout tourne sur **un seul PC Windows** : la vidéo est une capture d'écran de la fenêtre d'Arma, XERULITH doit donc être sur le même PC que le jeu. Deux écrans sont idéaux : Arma sur l'un, XERULITH sur l'autre.

9.2 Prérequis

Élément	Détail
Arma 3	Version Steam à jour (Windows 64 bits). Les cartes du jeu de base suffisent : Altis, Stratis, Malden
Mod Pythia	Steam Workshop, « Pythia » (s'abonner : le lanceur le télécharge), ou github.com/overfl0/Pythia
Fichiers du pont	Livrés avec XERULITH dans `%LOCALAPPDATA%\Programs\XERULITH\arma3\` : `@XerulithBridge\`, `mission_drone\init.sqf`, `mission_drone\xerulith_bridge.sqf` et `LISEZMOI.md`
BattlEye	**Désactivé** : Pythia n'est pas autorisé par BattlEye. Jeu en solo ou serveur local uniquement
XERULITH	Version 0.4.0 ou plus, profil **Drone Arma 3 (simulation de formation)**
IA (facultatif)	Serveur Ollama pour les missions autonomes (chapitre 8)

9.3 Installer les mods

1. Installez **Pythia** : abonnez-vous au mod sur le Steam Workshop d'Arma 3. Il apparaît ensuite dans l'onglet **Mods** du lanceur.
2. Copiez le dossier **@XerulithBridge complet** dans le dossier d'installation d'Arma 3, à côté de `arma3_x64.exe`. Avec Steam, c'est en général `C:\Program Files (x86)\Steam\steamapps\common\Arma 3\`.
3. Vérifiez la structure : les deux sous-dossiers sont indispensables.

```
Arma 3\
├── arma3_x64.exe
└── @XerulithBridge\
    ├── addons\
    │   └── xerulith_bridge.pbo     (addon minimal, CfgPatches)
    └── python_code\
        ├── $PYTHIA$                 (contient : XerulithArma)
        └── __init__.py              (le module du pont)
```

> Pythia ne cherche le code Python **que dans les mods dont Arma a chargé un fichier .pbo**. Sans `addons\xerulith_bridge.pbo`, la mission s'arrête sur « Pythia module "XerulithArma.poll" not recognized ». S'il manque, recopiez le dossier `@XerulithBridge` complet depuis `arma3` du dossier d'installation de XERULITH.

4. À chaque mise à jour de XERULITH, **remplacez le dossier @XerulithBridge entier** et les deux scripts de la mission : le protocole du pont évolue avec l'application (version affichée à la connexion).

9.4 Paramétrer le lanceur d'Arma 3

Ces réglages se font une fois ; le lanceur les garde pour les lancements suivants.

1. Ouvrez le **lanceur d'Arma 3** (Steam → Arma 3 → Jouer).
2. Onglet **Mods** : cliquez **Mod local** (ou « Ajouter un mod local »), choisissez le dossier `@XerulithBridge`, puis **cochez @Pythia et @XerulithBridge**. Le bandeau vert « Mod XerulithBridge installé » confirme l'ajout.
3. Onglet **Paramètres** (Parameters), section **Paramètres avancés** ou **Tous les paramètres** : cochez **Mode fenêtré** (Window mode).

4. Dans le champ de texte libre en bas de la liste (**Paramètres supplémentaires**, Additional parameters), tapez : `-noPause -noPauseAudio`.
5. Désactivez **BattlEye** (option BattlEye du lanceur), puis cliquez **JOUER AVEC MODS**.

Réglage	Pourquoi
`@Pythia` + `@XerulithBridge` cochés	Le pont Python est chargé avec le jeu
Mode fenêtré (`-window`)	La fenêtre du jeu reste visible à côté de XERULITH ; la capture d'écran lit sa zone client
`-noPause`	**Indispensable** : sans lui, Arma se met en pause dès que sa fenêtre perd le focus, et le drone se fige quand on clique dans XERULITH
`-noPauseAudio`	Le son continue lui aussi en arrière-plan
BattlEye désactivé	Pythia est refusé par BattlEye

Avec un raccourci ou un fichier `.bat` au lieu du lanceur, mettez tous les paramètres sur la ligne de commande, après un espace :

```
"C:\...\Arma 3\arma3_x64.exe" -mod="!Workshop\@Pythia;@XerulithBridge" -window -noPause -noPauseAudio -skipIntro -noSplash
```

> Conseils d'affichage : une fenêtre de 1280 × 720 à 1600 × 900 suffit et soulage la carte graphique ; placez-la sur un écran où **rien ne la recouvre**, pas même XERULITH. La capture recopie l'écran : une fenêtre posée par-dessus apparaît dans la vidéo.

9.5 Préparer la mission

1. Dans Arma, ouvrez l'**Éditeur** (Eden) et choisissez une carte (Altis, Stratis, Malden…).
2. Placez **une unité jouable** (un soldat BLUFOR, par exemple) à l'endroit de départ voulu, sur un terrain dégagé.
3. Enregistrez le scénario (**Scénario → Enregistrer**), par exemple sous le nom `xerulith`. Arma crée le dossier `Documents\Arma 3 - Other Profiles\<profil>\missions\xerulith.Altis\` (`Documents\Arma 3\missions\` avec le profil par défaut).
4. Copiez `init.sqf` et `xerulith_bridge.sqf` dans ce dossier, à côté de `mission.sqm`.
5. Lancez le scénario (**Jouer le scénario** depuis l'éditeur, ou Solo → Scénarios). Un drone AR-2 Darter apparaît 8 m devant le joueur, à **10 m de hauteur**, et la vue passe sur sa caméra, sans l'interface d'Arma (le HUD est celui de XERULITH). Le message « XERULITH : pont ok sur 127.0.0.1:47800, vol physique (modèle de vol Arma) » s'affiche.

Le pont se lance par une ligne de `init.sqf`. Ses paramètres :

```
[_uav, 47800, "127.0.0.1", "physique", "camera"] execVM "xerulith_bridge.sqf";
```

Paramètre	Valeurs	Rôle
`_uav`	objet	Le drone (ou un hélicoptère) à piloter ; il doit avoir son équipage IA (UAV de l'éditeur, ou `createVehicleCrew`)
port	`47800`	Port TCP du pont
hôte	`"127.0.0.1"`	Interface d'écoute ; `"0.0.0.0"` pour un XERULITH sur un autre PC (sans vidéo, § 9.12)
mode	`"physique"` (défaut), `"arcade"`	Modèle de vol d'Arma, ou vitesse imposée et drone tenu à plat (§ 9.8)
vue	`"camera"` (défaut), `"hud"`, `"conducteur"`	Caméra libre fixée au véhicule, sans l'interface d'Arma (HUD de XERULITH, § 9.7) ; vue de la tourelle avec le HUD d'Arma ; poste de conduite de

Paramètre	Valeurs	Rôle
		l'UGV (le pont conduit alors tout droit)

- **Hauteur de départ** : variable `_hauteur = 10` en tête de `init.sqf`.
- **Véhicule placé dans l'éditeur** : donnez-lui le nom de variable `xer_vehicule` (champ « Nom de variable ») : `init.sqf` le pilote à la place du drone qu'il crée (drone, hélicoptère ou UGV sans équipage).
- **Rover UGV** : utilisez les fichiers du dossier `mission_rover` (une deuxième mission, par exemple `xerulith_rover.Altis`) ou mettez `_vehicule = "ugv"` en tête de `init.sqf` : un UGV Stomper apparaît 10 m devant le joueur, au sol (§ 9.11).
- **Arrêter le pont** sans quitter la mission : console de débogage (Échap), `xer_bridge_run = false;` ; la vue revient au joueur.
- Les deux scripts se modifient avec un simple éditeur de texte ; la mission les relit à chaque lancement.

9.6 Connecter XERULITH

Lancer Arma 3 depuis XERULITH : avec un profil Arma 3 (drone ou rover), la barre de connexion affiche le bouton 🎮 **Lancer ARMA 3**. Il lance directement `arma3_x64.exe`, sans le lanceur et sans BattlEye, avec les mods `@Pythia` et `@XerulithBridge` et les paramètres `-window -noPause -noPauseAudio -skipIntro -noSplash` : les réglages des § 9.4 sont alors appliqués automatiquement. Arma 3 est cherché dans le registre et les bibliothèques Steam, Pythia dans le dossier d'Arma ou le contenu Workshop, `@XerulithBridge` dans le dossier d'Arma ou, à défaut, dans celui de XERULITH. Le journal affiche la ligne de commande. Steam doit pouvoir s'ouvrir.

Réglage du profil (⚙️ → Contrôleur)	Rôle
Chemin de arma3_x64.exe	Vide : recherche automatique ; sinon le chemin du programme ou de son dossier
Mission à ouvrir au lancement	Dossier de la mission (`…\missions\xerulith.Altis`) ou son `mission.sqm` : Arma l'ouvre dans l'éditeur Eden, il reste à cliquer **Jouer**
Paramètres de lancement	`-window -noPause -noPauseAudio -skipIntro -noSplash` par défaut ; `-noPause` est indispensable
Mods supplémentaires	Autres mods à charger, séparés par `;` (ex. `@CBA_A3`)

1. Choisissez le profil **Drone Arma 3 (simulation de formation)** dans la liste **Robot**.
2. L'adresse est `127.0.0.1:47800` (champ **Pont Arma 3**). Cliquez **Connecter** : le journal affiche « Connecté : Arma 3 : B_UAV_01_F sur Altis (pont 1.5) » et rappelle de lancer Arma avec `-noPause`.
3. Cliquez **Démarrer la caméra** : XERULITH cherche la fenêtre du programme `arma3_x64.exe` et le journal indique la fenêtre retenue, par exemple « Capture de la fenêtre … (arma3_x64.exe, 1280x720) ». L'Explorateur Windows et le lanceur sont écartés, même si leur titre contient « Arma 3 ».
4. Pilotez depuis XERULITH. Quand Arma a la souris, `Alt+Tab` (ou Échap dans Arma) la rend ; grâce à `-noPause`, le drone continue de voler pendant que vous cliquez dans XERULITH.

9.7 Piloter le drone

Commande	Effet en vol physique
Avancer / Reculer	Le pilote automatique rejoint un point situé à vitesse horizontale × durée du pas (3 m par défaut) ; le drone s'incline, accélère puis freine

Commande	Effet en vol physique
Gauche / Droite	Nouveau cap (45°/s × durée du pas), tenu en stationnaire
Monter / Descendre	Change la hauteur de vol (1,5 m/s × durée du pas) ; une descente sous 1,5 m est refusée
Pavé Tête	Oriente la caméra par pas de 8° (repère **CAM** sur la bande de cap du HUD) ; le bouton central la recentre (inclinaison −15°). La caméra suit le cap du drone
STOP (Échap)	Le drone freine avec son inertie et se stabilise en stationnaire
Clavier	Flèches ou ZQSD ; Page précédente / Page suivante pour monter et descendre

Le **bandeau de télémétrie** est incrusté en bas de l'image capturée :

Ligne	Contenu
1 (vert)	**HAUT** hauteur sol (m), **ALT** altitude (m), **VIT** vitesse (km/h), **CAP** cap vrai (°)
2 (jaune)	**TÉLÉMÈTRE** distance de l'obstacle devant (portée 150 m), **CARB** carburant, **DÉGÂTS**, mode de vol, « EN MOUVEMENT » pendant un trajet
3 (bleu)	Pendant un exercice SATER : **RADIOGONIO 121,5 MHz**, relèvement vrai et écart avec le nez du drone, niveau en dBm
—	« FLIR » s'ajoute à la ligne 2 quand la caméra thermique est active

HUD de XERULITH (panneau Caméra, case **HUD**, cochée par défaut) : dessiné par XERULITH sur la vidéo à partir de la télémétrie, il remplace l'interface d'Arma (vue `camera`, défaut). En haut, une **bande de cap** avec ses repères : **H** retour au départ, repère jaune vers la **cible** du trajet en cours, **#n** objets vus par le détecteur du rover, **121,5** balise SATER, **CAM** direction de la caméra quand elle est tournée. Au centre, un **horizon artificiel** (tangage, roulis ; rouge au-delà de 20° / 15° pour le rover). À gauche vitesse, tangage, roulis, hauteur sol, altitude ; à droite télémètre, carburant (jauge), dégâts, radiogoniomètre. En bas, position E / N, distance et cap du départ, distance de la cible. Le HUD est ajouté à l'affichage seulement : les images du pilote IA restent sans HUD. Il fonctionne aussi avec les autres sources vidéo (caméra IP, ESP32-CAM, webcam), avec les mesures que fournit le robot.

Les mêmes mesures sont dans l'onglet **Capteurs**, et le journal rappelle hauteur, cap et vitesse après chaque mouvement. Le bandeau incrusté en bas de l'image se coupe dans l'éditeur de profil (capteur caméra, « Télémétrie de vol incrustée dans l'image »).

9.8 Modes de vol

Mode	Fonctionnement
physique (défaut)	Le drone vole avec le **modèle de vol d'Arma**. Son pilote IA, mis en mode « sans combat », sert de pilote automatique : il rejoint chaque consigne comme le mode « guidé » d'un vrai multirotor. Le script ajoute un **maintien de hauteur** (boucle de vitesse verticale amortie, qui supprime le « pompage » de l'IA en stationnaire), un **maintien de position** doux (type stationnaire GPS, accélération bornée à 2 m/s²) et un **maintien de cap** en stationnaire. XERULITH attend que le drone soit stabilisé avant l'ordre suivant
arcade	Vitesse imposée à chaque image, drone tenu à plat, arrêt net. Utile pour des essais de capture ou de script, sans réalisme

Arma ne permet pas de commander directement les gaz et le cyclique d'un hélicoptère par script : en mode physique, le drone est donc piloté **par consignes** (point, hauteur, cap), ce qui correspond au fonctionnement d'un drone de reconnaissance en mode guidé.

9.9 Réglages du profil drone

Réglage (éditeur de profil)	Défaut	Rôle
Adresse du pont	`127.0.0.1:47800`	Contrôleur Arma 3
Origine latitude / longitude	48,5436 / 3,0036	Les mètres de la carte sont convertis en latitude / longitude autour de cette origine (capteur Position)
Vitesse horizontale	3 m/s	Pas Avancer / Reculer ; jusqu'à 60 m/s acceptés par le pont
Vitesse verticale	1,5 m/s	Monter / Descendre
Vitesse de rotation	45 °/s	Gauche / Droite
Durée d'un pas	1 s	Un pas = 3 m ; 2 s donnent des pas de 6 m, plus naturels
Vitesse de transit	0 (maximale)	Trajets ALLER / VOLER sans vitesse précisée ; 0 = vitesse maximale du drone
Rayon maximal	6000 m	Zone autorisée autour du point de départ ; un ALLER hors zone est refusé
Hauteur maximale	150 m	HAUTEUR de 5 m à cette valeur
Distance d'arrêt	500 cm	Exploration : AVANCER refusé si le télémètre mesure moins
Caméra : fenêtre, ips, largeur	« Arma 3 », 15, 960	Programme ou titre de la fenêtre capturée, images par seconde, largeur de l'image
Tête : centre d'inclinaison, balayage	−15°, 40°	Position de repos de la tourelle ; panorama gauche / centre / droite de l'exploration

9.10 Missions autonomes du drone

Dans l'onglet **Exploration autonome**, le pilote reçoit, en vol physique, des directives de navigation en plus de AVANCER, RECULER, TOURNER, MONTER et DESCENDRE :

Directive	Effet
`ALLER: <est> <nord> [vitesse]`	Rejoint le point situé à est et nord mètres du **point de départ** (`ALLER: -500 1000`, `ALLER: -500 1000 12` à 12 m/s). Ouest et sud en négatif
`ALLER: depart [vitesse]`	Revient au point de départ
`VOLER: <cap> <distance> [vitesse]`	Vole la distance au cap vrai (`VOLER: 245 200`, `VOLER: 245 200 54 km/h`)
`HAUTEUR: <m>`	Hauteur sol, de 5 m à la hauteur maximale du profil
`VITESSE: <m/s> | max`	Vitesse des trajets suivants (`VITESSE: 15`, `VITESSE: 30 km/h`, `VITESSE: max`) ; ne compte pas de pas
`FLIR: on|off`	Caméra thermique, blanc = chaud ; chaque observation signale alors les points chauds et leur position au sol
`SUIVANT`	Rejoint le prochain point du plan de vol préparé par XERULITH (missions incendie et rover)

- Le **point de départ** est la position du drone au lancement de la mission (bouton ▶️ Démarrer).
- Chaque observation donne la position **mesurée** : est et nord en mètres, distance et cap pour revenir au départ, cap actuel, hauteur ; puis le radiogoniomètre pendant un exercice SATER. La carte du panneau trace le vol réel, nord en haut.
- Vitesse d'un trajet : celle de la directive, sinon la dernière `VITESSE:`, sinon la vitesse de transit du profil (maximale par défaut).
- Un ALLER, un VOLER ou un HAUTEUR compte pour un pas du budget ; STOP interrompt le trajet, le drone freine.

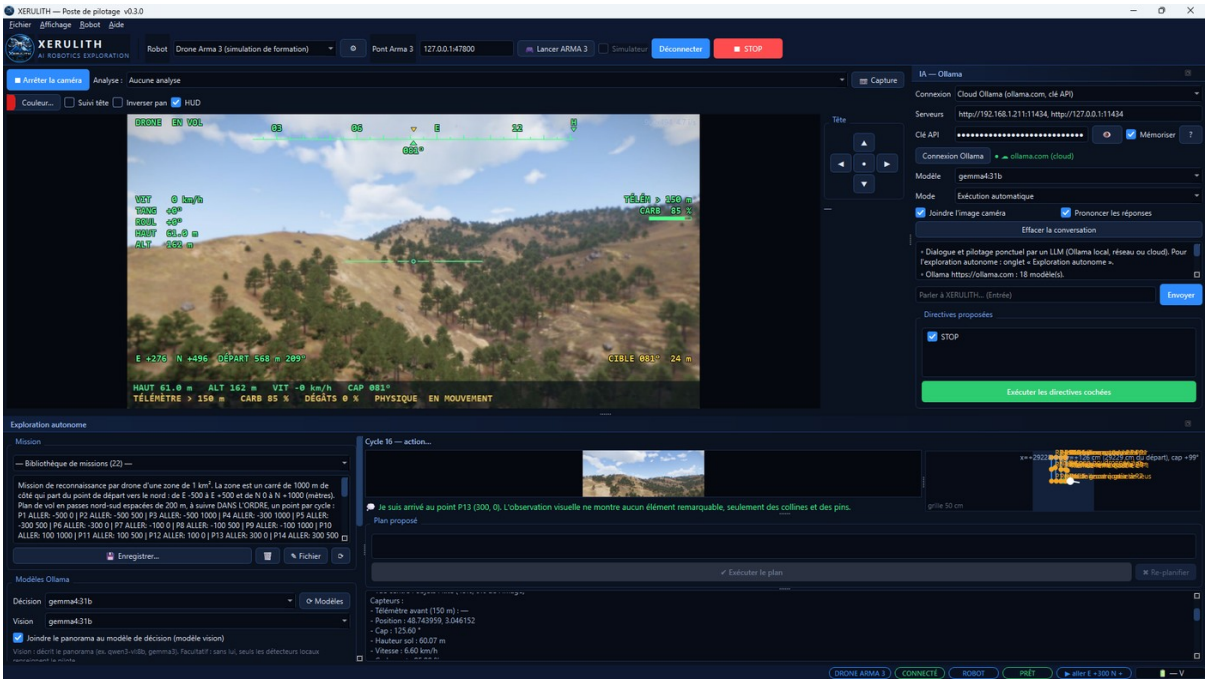
Procédure conseillée :

1. Pont connecté, caméra démarrée, drone stable en stationnaire.
2. Choisissez la mission dans la catégorie **Drone Arma 3 (formation)** et réglez les limites du panneau (tableau ci-dessous).
3. Premier essai en mode **Confirmer chaque plan** : vous validez chaque trajet proposé. Passez en **Autonome** une fois la mission rodée.
4. ▶️ **Démarrer**. Le rapport de mission (bouton 📄 Rapport) est rédigé à la fin.

Mission	Cycles max	Pas max	Durée max
Reconnaissance 1 km² et retour	40	60	45 min
SATER : recherche de balise 121,5 MHz	60	80	40 min
Surveillance incendie FLIR 20 km²	70	80	120 min
Exploration terrestre rover UGV 1 km² (§ 9.11)	90	150	60 min

Reconnaissance 1 km² et retour

La zone est un carré de 1000 m de côté au nord du départ : de E −500 à E +500 et de N 0 à N +1000. Le prompt donne au pilote un plan de vol de **6 passes nord-sud espacées de 200 m**, à 60 m de hauteur, soit 18 points (P1 `ALLER: -500 0`, P2 `ALLER: -500 500`, P3 `ALLER: -500 1000`, P4 `ALLER: -300 1000`… P18 `ALLER: 500 0`), puis le retour au départ. À chaque point, le pilote note chaque élément remarquable (bâtiment, véhicule, personne, route, pont, cours d'eau, feu, fumée, zone détruite, obstacle au vol) et écrit « NOTE: P<n> fait » pour suivre sa progression. Un obstacle à moins de 50 m devant lui fait prendre 90 m de hauteur. La mission se termine par HAUTEUR: 10 au départ et un bilan des éléments repérés avec leur position.



Reconnaissance 1 km² : texte de la mission et plan de vol, pilote IA en exécution automatique, cible du trajet sur le HUD (CIBLE 081°, 24 m).

SATER : recherche de balise 121,5 MHz

Au lancement de la mission, XERULITH demande au pont de placer **au hasard** une épave d'hélicoptère civil, un blessé et une balise de détresse ELT 121,5 MHz, entre 300 m et le rayon cité par la mission (1500 m), sur un terrain plat hors de l'eau. **La position reste cachée** au pilote et à l'opérateur. Le drone porte un **radiogoniomètre simulé** :

Grandeur	Modèle
Niveau reçu	PIRE +17 dBm, affaiblissement en espace libre (20 log d + 74,1 dB, d en km), ±3 dB de bruit ; **−22 dB** si le relief masque la balise
Ordres de grandeur	−60 dBm à 1500 m, −50 à 500 m, −40 à 150 m, −30 à 50 m, −23 à 20 m
Relèvement	Cap vrai vers la balise, ±10° en vue directe, ±35° derrière le relief (réflexions), très instable à moins de 40 m (à l'aplomb)

Le prompt enseigne la méthode du **ralliement par relèvements successifs** : monter à 80 m pour capter au-dessus du relief ; bonds de 300 m au relèvement sous −50 dBm, de 100 m entre −50 et −40 dBm ; au-dessus de −40 dBm, descendre à 30 m, bonds de 40 m et regard vers le bas ; un relèvement qui bascule de plus de 90° signale le passage à la verticale de la balise ; sans signal, ou si le niveau baisse deux fois, remonter à 120 m. Le pilote note chaque relèvement pour recouper. Épave vue ou niveau au-dessus de −30 dBm : il note la position, annonce « Balise localisée », revient au départ et termine.

Évaluation de l'instructeur : le rapport de mission ajoute la distance finale du drone à la balise, la plus courte distance atteinte et la réussite (drone passé à moins de 50 m), calculées sur la position vraie. Dans Arma, la console de débogage révèle le repère caché sur la carte : `"xer_balise_mk" setMarkerAlphaLocal 1;`. Une seule balise par mission Arma : pour un nouvel exercice, relancez la mission Arma puis reconnectez XERULITH.

Surveillance incendie FLIR 20 km²

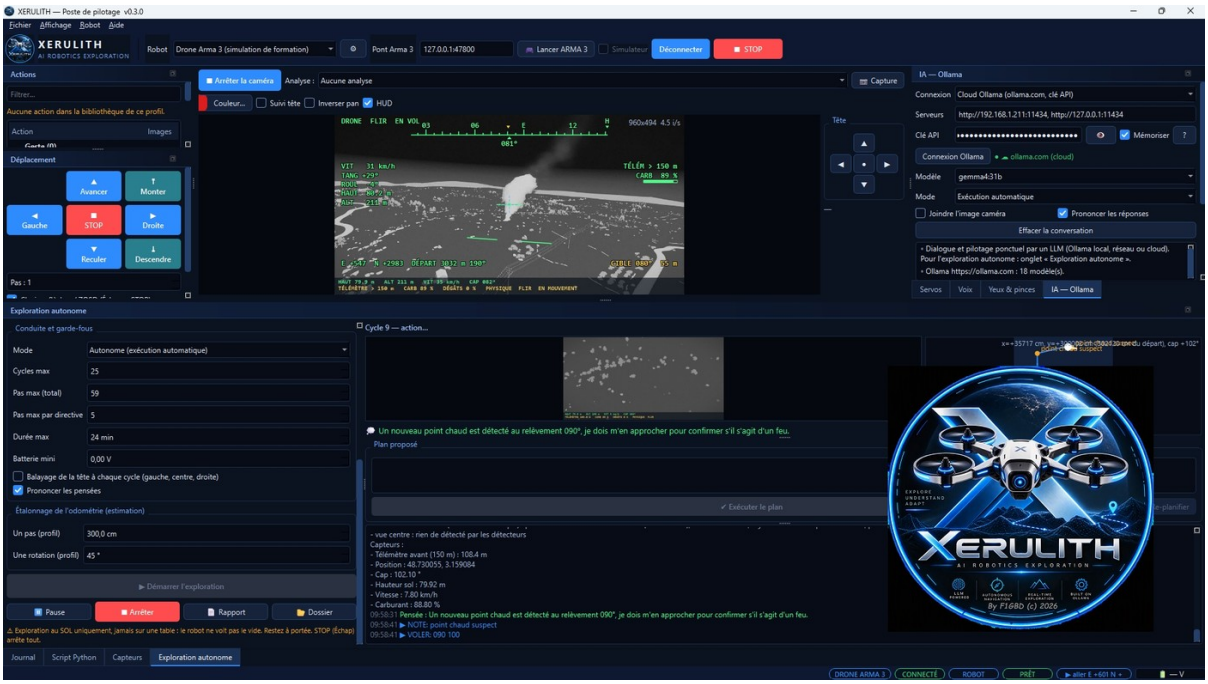
Au lancement, XERULITH fait placer **au hasard** un départ de feu (foyer avec fumée et véhicule en feu) dans la zone citée par la mission : un rectangle de 5 km sur 4 km au nord du départ (E −2500 à +2500 m, N 0 à +4000 m). La position reste cachée. Le pilote passe en caméra thermique (`FLIR: on`), monte à 150 m et vole à **30 km/h** (`VITESSE: 30 km/h`). **XERULITH prépare lui-même le plan de vol** : quadrillage en **5 passes nord-sud espacées de 1000 m**, un point tous les 500 m, points en mer retirés. Le pilote n'a aucun point à calculer : il répond `SUIVANT` pour rejoindre le prochain point, et l'observation lui rappelle « k/N points faits » et le prochain point. Les réponses du modèle restent courtes et la mission démarre aussitôt.

Feu toujours détectable en formation : le feu est placé au hasard, à terre, **près d'un point de passage** du plan de vol, entre ce point et le suivant de la même passe. Le drone arrive dans l'axe et le foyer est devant lui, à moins de 500 m.

Détection : un foyer ne fait souvent que quelques pixels à 150 m de hauteur. Le pont (version 1.7) embarque donc un **détecteur thermique simulé**, comme la boule FLIR d'un vrai drone : il signale le foyer quand il est dans le champ de la caméra (position dans l'image) et le **panache chaud** au-dessus de lui jusqu'à 3 km, même quand le relief cache le foyer. À chaque point, XERULITH interroge le détecteur vue par vue (gauche, centre, droite) ; sans lui (ancien pont), il cherche les taches blanches de l'image thermique.

- **Foyer visible mais hors de l'image** (sous le drone, sur le côté) : XERULITH **pointe automatiquement la caméra** sur le foyer à partir du cap, du relèvement, de la distance et de la hauteur, puis refait la mesure.
- **Foyer caché par le relief** : l'observation donne le relèvement et la distance du panache, avec la directive pour s'en approcher (`VOLER: <cap> <distance>`).

- **Foyer dans l'image** : la caméra reste pointée dessus et le pont donne sa **position au sol**. L'observation contient « ALERTE POINT CHAUD … position estimée au sol : E, N, latitude, longitude (en degrés, minutes, secondes), distance, relèvement », avec la directive `ALLER` pour s'en approcher.



Caméra thermique (FLIR) : le panache chaud du départ de feu est détecté au relèvement 090° ; le pilote note « point chaud suspect » et s'en approche (VOLER: 090 100).

Point chaud réel enregistré : dès qu'une position au sol est mesurée, XERULITH joue une **tonalité d'alerte** (trois bips sur le PC), prend une **photo de la zone** annotée (cercle sur le point chaud, bandeau avec heure, cycle, E / N, coordonnées en degrés, minutes, secondes et décimales, distance) et enregistre ses coordonnées. Dans le dossier de la mission : `point_chaud_NN.jpg` et `points_chauds.csv` (n°, heure, cycle, E, N, latitude, longitude, latitude et longitude DMS, distance, relèvement, source, photo). Un point chaud à moins de 60 m d'un point déjà enregistré n'est pas répété. Le rapport ajoute une section « Points chauds détectés ».

Le pilote s'en approche à 80 m, caméra vers le sol (`REGARDER: bas`), confirme le feu, le signale (`NOTE: FEU CONFIRMÉ`, la position mesurée est ajoutée à la note ; `DIRE`), rentre et termine avec la position du feu. Au retour (`ALLER: depart`) et en fin de mission, la caméra repasse en **vision normale**.



Confirmation en vision normale : foyer et fumée du départ de feu sous le drone, HUD de XERULITH (hauteur 150 m, cap 333°, position E +63 m, N +3271 m).

- **Pas de survol de la mer** : il n'y a pas de risque de feu en mer. Au lancement, XERULITH relève une carte terre / mer de la zone (maille de 100 m) ; le feu n'est placé qu'à terre, les points en mer sont retirés du plan de vol, un ALLER vers la mer est refusé et un trajet qui couperait la mer est automatiquement contourné par la terre, avec une marge de 100 m le long des côtes.
- Réglez le panneau sur **70 cycles, 80 pas, 120 min** et le **rayon maximal** du profil sur au moins 5000 m (6000 m par défaut) : le coin le plus éloigné de la zone est à 4,5 km du départ.
- `FLIR: on` : la caméra du véhicule (vue `camera`, défaut) passe directement en thermique, le HUD de XERULITH reste affiché. En vue tourelle (`hud`) : Arma ne permet pas de passer la tourelle en thermique par script ; le pont bascule donc sur une **caméra thermique** fixée sous le nez du drone, orientée comme la tourelle (le HUD d'Arma disparaît, le bandeau de XERULITH reste). `FLIR: off` rend la vue tourelle et son HUD. Une heure matinale ou nocturne (Eden : Attributs → Environnement) donne le meilleur contraste.
- Toute mission dont le texte cite « départ de feu » (ou « incendie ») et « FLIR » (ou « thermique ») déclenche le placement du feu ; la zone se lit dans le texte (« de E -2500 à E +2500 et de N 0 à N +4000 »).
- Le carburant du drone baisse pendant ce long vol : la mission ramène le drone sous 20 %. Pour un exercice sans contrainte, console de débogage : `xer_uav setFuel 1`.
- Évaluation instructeur dans le rapport : position réelle du feu, écart du meilleur signalement, **feu localisé** sous 150 m. Repère caché : `"xer_feu_mk" setMarkerAlphaLocal 1;`.

Écrire une mission de drone

- Donnez les points en **mètres est / nord depuis le départ** : les petits modèles ne savent pas calculer des caps et des distances de tête.
- **Un trajet par cycle**, puis une observation ; demandez une `NOTE` à chaque étape pour que le pilote sache où il en est.
- Fixez la vitesse dès le premier cycle (`VITESSE: 15`) ou point par point (`ALLER: 300 0 8` pour une passe lente d'observation).
- Toute mission dont le texte cite « balise » et « 121,5 » déclenche le placement de la balise ; le rayon se lit dans le texte (« dans un rayon de 2 km »), dans la limite du rayon maximal du profil.
- Terminez toujours par `ALLER: depart`, une hauteur basse et `FIN:` avec le bilan attendu.

9.11 Rover UGV : exploration terrestre

Le pont reconnaît un **véhicule terrestre** (un UGV Stomper `B_UGV_01_F` par défaut) et passe en **conduite** : le rover roule toujours avec la physique d'Arma (suspension, adhérence, pente), sans hauteur de vol. Il simule un rover d'exploration : trajets en tout-terrain, mesure de la pente, détection des obstacles et des objets.

1. Préparez une mission avec les fichiers du dossier `mission_rover` (§ 9.5) et lancez-la : l'UGV apparaît 10 m devant le joueur et la vue passe sur sa caméra avant ; le HUD est celui de XERULITH.
2. Dans XERULITH, choisissez le profil **Rover UGV Arma 3 (simulation de formation)**, adresse `127.0.0.1:47800`, **Connecter** : le journal affiche « Arma 3 : B_UGV_01_F sur Altis (pont 1.5) ».
3. **Démarrer la caméra** comme pour le drone, puis pilotez au pavé ou lancez une mission autonome.

Commande	Effet pour le rover
Avancer / Reculer	Vitesse imposée pendant la durée du pas (profil : 2 m/s × 1,5 s = 3 m)
Gauche / Droite	Rotation sur place (30 °/s × 1,5 s = 45°)
Monter / Descendre, `HAUTEUR`	Refusés
`ALLER: <est> <nord> [vitesse]`	Trajet de l'**IA conductrice** d'Arma : elle choisit le chemin en tout-terrain et contourne les obstacles ; arrivée à 12 m du point
`ROULER: <cap> <distance>`	Comme `VOLER` pour le drone (`ROULER: 245 80` : 80 m au cap

Commande	Effet pour le rover
`[vitesse]`	245°)
`SUIVANT`	Prochain point du parcours préparé par XERULITH
Pavé Tête	Oriente la tourelle caméra (repos : inclinaison −5°, balayage ±60° en exploration)

- **Télémétrie** : position, cap, vitesse, **tangage** et **roulis** (la pente du terrain), télémètre avant (150 m, à 0,6 m du sol), carburant, dégâts. Le bandeau de l'image affiche « ROVER », « BLOQUÉ » et le nombre d'objets en vue du détecteur.
- **Conduite** : l'**IA conductrice** d'Arma conduit le rover et cherche le chemin (vue `camera`, défaut). Si elle reste immobile 10 s au début d'un trajet ou renonce, le pont prend le relais en **conduite directe** (cap vers le point, vitesse demandée bornée à 14 m/s, arrêt à 12 m ; message « conduite directe du rover par le pont »). La vue `conducteur` (poste de conduite avec le HUD d'Arma) impose la conduite directe : l'IA ne roule pas quand on regarde par sa caméra.
- **HUD de XERULITH** pour le rover : horizon artificiel (rouge au-delà de 20° de tangage ou 15° de roulis), repères **#n** des objets détectés et repère jaune de la cible du trajet sur la bande de cap.
- **Trajet bloqué** : rover immobilisé loin du point (20 s avec l'IA, 12 s en conduite directe), IA conductrice qui renonce ou arrêt à plus de 25 m du point. XERULITH recule alors de 15 m et tente seul un **contournement** par la gauche puis par la droite (point décalé de 100 m) avant de rendre la main au pilote.
- **Alerte pente** dans l'observation au-delà de 20° de tangage ou de 15° de roulis : risque de retournement, le pilote change de direction.
- Le rover ne roule pas dans l'eau : même carte terre / mer que la mission incendie ; les points en mer sont retirés du parcours.
- Réglages du profil : vitesse de transit 4 m/s (≈ 15 km/h), rayon maximal 3000 m, distance d'arrêt 3 m.

Exploration terrestre rover UGV 1 km²

Au lancement, XERULITH demande au pont de déposer **au hasard**, à terre et sur un sol peu pentu, **5 objets à identifier** dans la zone (E −500 à +500 m, N 0 à +1000 m) : caisse de matériel, épave de véhicule, tente, fûts métalliques, terminal de données, pile de palettes, projecteur de chantier… Les positions restent cachées. XERULITH prépare le **parcours** : 7 passes nord-sud espacées d'environ 143 m, un point tous les 250 m (35 points), points en mer retirés ; le pilote répond `SUIVANT`.

Un **détecteur d'objets embarqué**, simulé par le pont, signale les objets que la caméra voit (moins de 80 m, dans ±50° de son axe, sans obstacle entre les deux) : l'observation donne le numéro, la nature, la distance, le relèvement, la position estimée et la directive `ALLER` pour s'en approcher. À moins de 15 m, le pilote identifie l'objet par `NOTE: SITE <n> identifié : <nature>` ; XERULITH ajoute à la note la position mesurée de l'objet (E, N, latitude, longitude). La mission se termine quand les 5 objets sont identifiés ou le parcours terminé : retour au départ et bilan.

- Réglez le panneau sur **90 cycles, 150 pas, 60 min**.
- Toute mission de rover dont le texte cite « objets à identifier » (ou « exploration ») déclenche le placement ; la zone (« de E -500 à E +500 et de N 0 à N +1000 ») et le nombre d'objets (« 5 objets ») se lisent dans le texte.
- **Évaluation instructeur** du rapport : tableau des objets (position réelle, vu ou non, plus courte distance du rover, signalé ou non, écart de position), parcours fait, trajets bloqués, distance parcourue, distance finale au départ, plus forte inclinaison. Repères cachés dans Arma : `"xer_site_mk_1" setMarkerAlphaLocal 1;` (2, 3…).

> Pont 1.5 : pour le rover, recopiez `@XerulithBridge\python_code` dans le dossier d'Arma et `xerulith_bridge.sqf` dans vos missions existantes. Les missions du drone fonctionnent avec les deux versions.

9.12 Formation et sécurité du pont

- Le pont n'écoute que sur `127.0.0.1` : seul un XERULITH du même PC peut piloter. Avec l'hôte `"0.0.0.0"`, un XERULITH d'un autre PC du réseau de formation peut piloter et recevoir la télémétrie, **mais pas la vidéo** (capture locale).
- Le pont n'accepte que douze ordres (vitesse, fin de pas, point de la carte, balise, feu, objets à identifier, carte terre / mer, thermique, pointage, arrêt, caméra, bonjour), bornés : 60 m/s, 20 m/s vertical, 120 °/s, 30 s par ordre, hauteur 2 à 500 m. Rien d'autre n'est exécuté.
- Un client perdu arrête le drone ; un ordre de vitesse expire seul ; STOP passe avant tout ordre en attente.
- Un poste par stagiaire : chaque PC fait tourner sa propre mission Arma et son XERULITH.

9.13 Dépannage Arma 3

Symptôme	Cause et solution
« Pythia module "XerulithArma.poll" not recognized »	`@XerulithBridge` non chargé (le cocher dans le lanceur) ou sans `addons\xerulith_bridge.pbo`. Recopier le dossier complet ; la liste des modules trouvés est dans le journal de Pythia (dossier d'Arma)
« XERULITH : Pythia n'est pas chargé »	`@Pythia` non coché, ou BattlEye actif
« Pont Arma 3 injoignable »	Mission non lancée, script non copié, port 47800 pris par un autre programme
« Le pont répond mais n'envoie aucune télémétrie »	Jeu en pause : lancer Arma avec `-noPause`, fermer le menu Échap
Le drone ne bouge plus quand on clique dans XERULITH	`-noPause` absent des paramètres du lanceur
« Fenêtre Arma 3 introuvable »	Fenêtre réduite, ou ancienne version de XERULITH ; vérifier le message « Capture de la fenêtre … » du journal
La vidéo montre une autre fenêtre ou des morceaux de XERULITH	Une fenêtre recouvre celle d'Arma : la laisser visible, idéalement sur un autre écran
Oscillations en stationnaire	Scripts d'une ancienne version : recopier `xerulith_bridge.sqf`
« ALLER, VOLER et HAUTEUR exigent le vol physique »	Pont lancé en mode `"arcade"` ; remettre `"physique"` dans `init.sqf`
« Point hors de la zone autorisée »	Le point dépasse le rayon maximal du profil (6000 m par défaut) ; augmentez-le dans l'éditeur de profil
Pas d'image thermique après FLIR: on	Script de mission d'une version antérieure : recopier le dernier `xerulith_bridge.sqf` dans la mission
Aucune ALERTE POINT CHAUD alors que le feu est visible	Pont antérieur à la version 1.7 (pas de détecteur thermique) : recopier `@XerulithBridge` et le dernier `xerulith_bridge.sqf`. Ancien pont : image saturée (préférer l'aube ou la nuit), feu trop petit à l'image (voler plus bas)
« foyer hors du champ de la caméra » en boucle	Version de XERULITH antérieure au pointage automatique de la caméra : mettre à jour XERULITH
« Pont Arma 3 trop ancien, version … requise »	La mission utilise un ancien `xerulith_bridge.sqf` : recopier celui de `arma3\mission_drone` (ou `mission_rover`, `mission_robot`) du dossier d'installation dans chaque mission, y compris celles des profils Arma (Documents\Arma 3 - Other Profiles\<profil>\missions)
Impossible de reprendre la souris	**Ctrl+Alt+X** ramène XERULITH au premier plan et libère la

Symptôme	Cause et solution
une fois Arma lancé	souris ; **Ctrl+Alt+S** donne le STOP depuis le jeu
Erreur de script dans Arma	Lire le dernier fichier `.rpt` de `%LOCALAPPDATA%\Arma 3\` ; le message donne la ligne du script

10. Robot humanoïde IA dans Arma 3

Le profil **Robot humanoïde Arma 3 (simulation de formation)** pilote un **robot humanoïde à pied** dans Arma 3, par le même pont que le drone et le rover (chapitre 9). Il sert à quatre exercices : **recherche de victimes** après un SATER en zone contaminée ou hostile, **reconnaissance de zone** avec mémoire du terrain, **exploration et découverte libre**, et **SATER au sol** à l'antenne Yagi.



Le robot humanoïde IA XERULITH : modèle 3D original du mod @XerulithRobot, sans arme.

10.1 Préparer le robot

1. Installez les mods comme pour le drone (§ 9.3) ; le dossier `arma3` de XERULITH contient aussi le mod du robot, `@XerulithRobot`, et la mission `mission_robot`.
2. Copiez `mission_robot` dans vos missions Arma 3 comme `mission_drone` (§ 9.5) : son `init.sqf` crée le robot (`_vehicule = "robot"`) devant le joueur.
3. Dans XERULITH, choisissez le profil **Robot humanoïde Arma 3 (simulation de formation)**. L'option du profil **« Robot humanoïde : modèle 3D IA XERULITH »** (⚙️ → Contrôleur) fait charger `@XerulithRobot` par « Lancer ARMA 3 » ; décochée, ou mod absent, la mission utilise un **mannequin** du jeu de base repeint en blanc et noir, sans aucun mod.
4. Adresse `127.0.0.1:47800`, **Connecter**, **Démarrer la caméra** : c'est la caméra de tête du robot (1,62 m), orientable au pavé Tête.

Commande	Robot humanoïde
Avancer / Reculer / pas de côté	Point à rejoindre à pied (1,2 m/s pendant 2 s : 2,4 m)
Gauche / Droite	Rotation sur place, 45° par pas
Monter / Descendre, `HAUTEUR`	Refusés
`ALLER` / `ROULER`	Trajet à pied choisi par l'IA d'Arma (chemins, obstacles, végétation) ; arrivée à 4 m
`VITESSE`	Marche jusqu'à 1,8 m/s, pas rapide jusqu'à 3,5 m/s, course au-delà

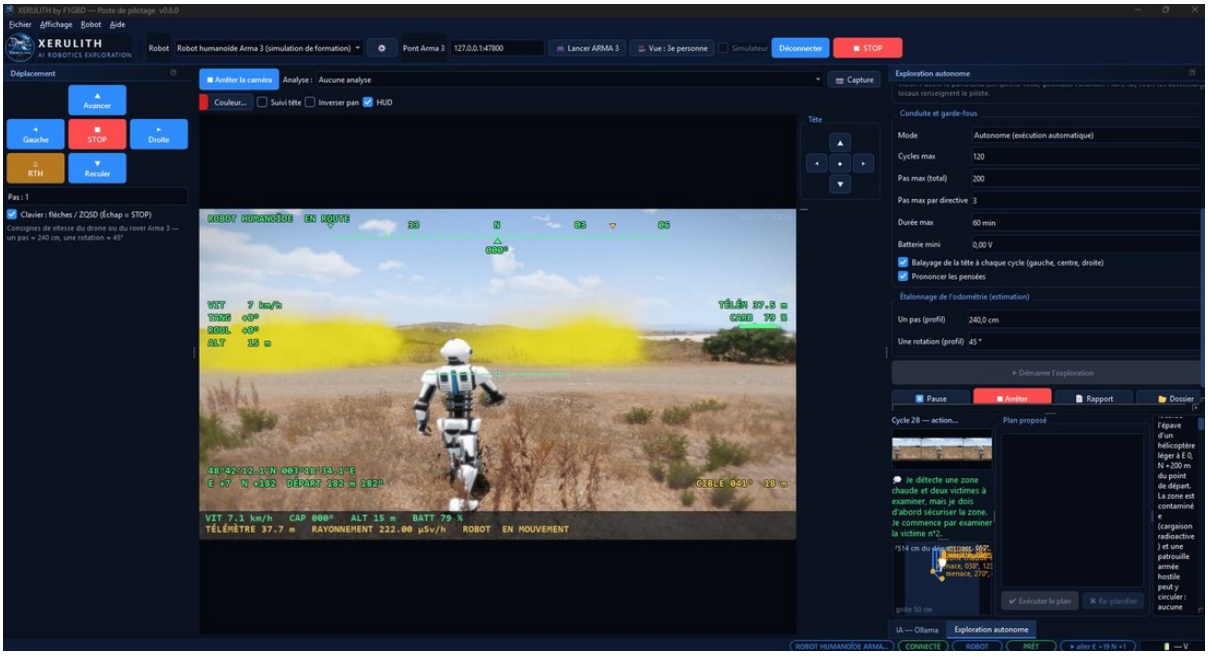
- **Batterie simulée** : environ 1 h de marche, 4 h à l'arrêt. **Trajet bloqué** : XERULITH tente seul un petit contournement (25 m) par la gauche, puis par la droite.
- **Vue extérieure** : le robot démarre en vue à la 3e personne (bouton 🎥 Vue, ou touche **Fin** dans Arma : 3e personne, orbite, caméra du robot). Pendant une mission, XERULITH repasse sur la caméra de tête le temps de chaque prise de vue du pilote IA : l'image bascule brièvement, c'est normal.

- Réglage conseillé du panneau Exploration : **120 à 150 cycles, 200 à 250 pas, 60 à 75 min** selon la mission (indiqué pour chacune ci-dessous).

10.2 Recherche de victimes (zone contaminée ou hostile)

Mission **« Robot humanoïde : recherche de victimes après un SATER (zone contaminée et hostile) »** (120 cycles, 200 pas, 60 min). Le drone a localisé l'épave ; au lancement, XERULITH place l'épave et **4 victimes au hasard** dans un rayon de 120 m, inconscientes ou conscientes et assises.

- **Zone contaminée** (le texte cite « contaminée ») : source de rayonnement à l'épave, fumée jaune ; le **détecteur de rayonnement** du robot donne le débit en µSv/h et XERULITH calcule la dose reçue.
- **Zone hostile** (le texte cite « hostile ») : une patrouille armée circule autour ; le robot est neutre, le **détecteur de personnes armées** les signale et le rapport compte les fois où il a été repéré.
- XERULITH prépare un **parcours** autour de l'épave ; le **détecteur de personnes** signale chaque victime en vue (moins de 80 m) ; le robot l'approche, et le pilote IA l'examine à l'image et note son état (`NOTE: VICTIME n : consciente ou inconsciente, …`).
- Fin : retour au départ et **compte rendu pour les secours** (victimes, zones chaudes, menaces, chemin d'accès conseillé), puis **évaluation instructeur** : victimes vues, signalées, écart de position, état juste ou à revoir, dose reçue.



Robot humanoïde en zone contaminée (fumée jaune) : débit de dose mesuré en continu, victimes à examiner.

10.3 Reconnaissance et exploration de zone

Mission **« Robot humanoïde : reconnaissance et exploration de zone (mémoire de terrain) »** (150 cycles, 250 pas, 75 min) : le robot découvre une zone inconnue (400 × 400 m au nord du départ par défaut), en dresse la **carte commentée** et s'en souvient pour les missions suivantes.

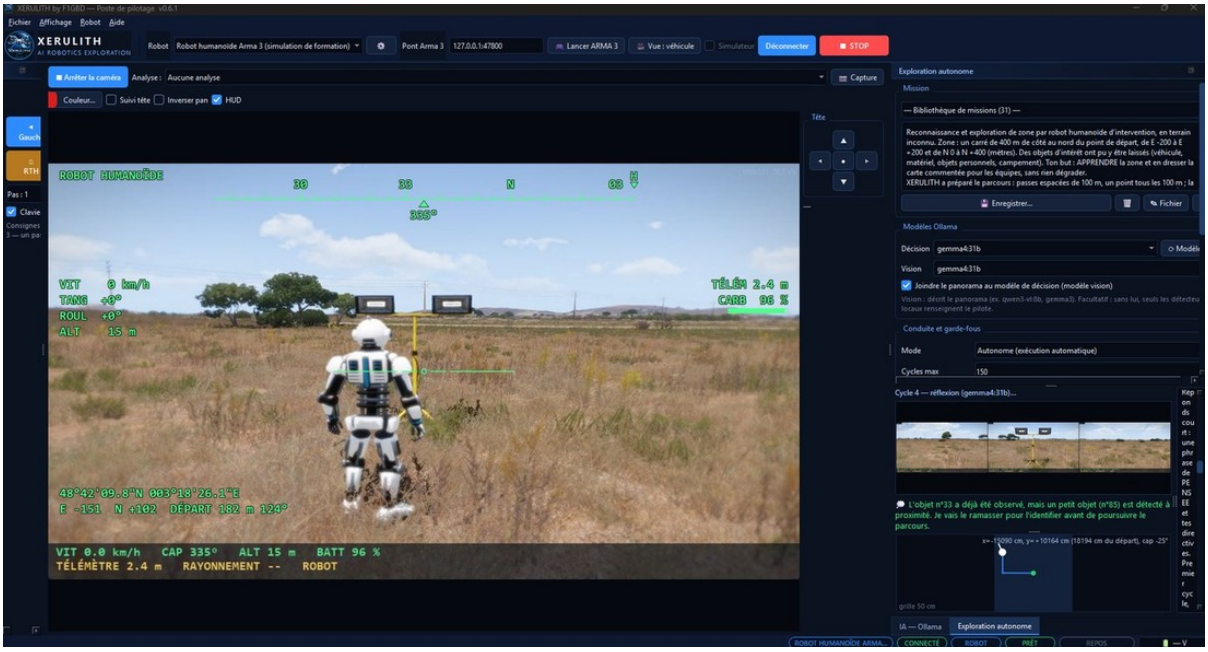
Le **détecteur d'objets embarqué** (simulé : lidar thoracique et vision) liste ce qui est en vue à moins de 60 m, avec un numéro qui ne change pas : véhicules, bâtiments, petits objets, éléments remarquables, personnes. XERULITH classe chaque élément par **intérêt** et propose la directive au pilote IA :

Directive	Déroulement
`OBSERVER: n`	S'approcher, pointer la caméra ; examen (dimensions, état, contenu, rayonnement), description par le modèle vision, photo
`RAMASSER: n`	Petit objet : le saisir, le **tenir devant la caméra en le tournant** (trois vues), le décrire, puis le **remettre à sa place**

Directive	Déroulement
`ENTRER: n`	Bâtiment : portes ouvertes si elles ne sont pas verrouillées, visite de 3 positions intérieures, sortie par l'entrée
`EMBARQUER: n`	Véhicule terrestre **non armé**, une fois par mission : environ 150 m au volant, regard depuis la cabine, puis descente
`QUITTER`	Descendre du véhicule, sortir du bâtiment ou reposer l'objet tenu

Garde-fous : jamais de contact avec une arme, des munitions ou un engin suspect (observation seulement) ; une personne s'observe sans la toucher ; pas de saisie au-dessus de 100 µSv/h ; pas de véhicule armé, verrouillé ou occupé ; nombre limité d'embarquements, de bâtiments, d'objets en main et d'observations par mission.

Avec « 6 objets d'intérêt » dans le texte, XERULITH dépose des objets à découvrir (véhicule civil, valise, ordinateur, radio, jerrican…, certains dans les bâtiments) et le rapport note ceux qui ont été vus et examinés ; sans cette mention, la zone est explorée telle qu'elle est.



Reconnaissance de zone : le robot s'approche d'un objet pour l'examiner ; la mémoire du terrain garde ce qu'il a appris.

10.4 Mémoire de terrain, curiosité et progrès

Tout ce que le robot voit et examine est inscrit, avec sa position, dans la **mémoire de terrain** : `xerulith_data\memoire_terrain\<carte>.json`, un fichier par carte d'Arma (§ 15). À la mission suivante :

- ce que la mémoire sait de la zone et les **leçons** des missions précédentes sont donnés au pilote IA dès le départ ; en repassant à moins de 90 m d'un endroit connu, ce qui y a été vu revient dans l'observation ;
- un élément **déjà connu et inchangé** n'est pas réexaminé ; un élément qui **a changé de place** est signalé et mérite un nouveau regard ;
- **curiosité** : un élément **jamais vu sur cette carte** est marqué « NOUVEAU » et passe avant le reste ; un élément déjà examiné souvent ailleurs intéresse moins ;
- à la fin de la mission, le pilote IA écrit 3 à 5 **leçons** (règles générales sur la zone et la méthode), gardées pour la suite.

Progrès d'une mission à l'autre : chaque mission enregistre son bilan (durée, distance, éléments examinés, déjà connus, réexamens évités, découvertes). Le rapport ajoute la section **« Progrès »** :

tableau des missions faites au même endroit et comparaison avec la première. Ce qui est appris est dans la mémoire de terrain ; le modèle d'IA, lui, ne change pas.

La recherche dans la mémoire se fait **par le sens** si le modèle d'embeddings d'Ollama est installé (`ollama pull nomic-embed-text`), sinon par mots-clés. Les effets d'ambiance du jeu (pollen, insectes, vent…) sont ignorés, et le détecteur de visages est coupé dans Arma. Supprimer le fichier de la carte efface la mémoire.

10.5 Exploration et découverte libre

Mission **« Robot humanoïde : exploration et découverte libre »** (150 cycles, 250 pas, 75 min) : la directive première du robot est de **découvrir des choses nouvelles et d'aller vers elles pour en apprendre plus**. Pas de zone ni de parcours imposés, pas de limite de distance au départ : seule la batterie le ramène.

- **Carte de ce qui a été vu** : le terrain est découpé en mailles de 25 m, et ce que le robot voit autour de lui (45 m) est marqué à chaque position. Elle est gardée dans la mémoire de terrain : la mission suivante sait où le robot est déjà allé.
- **Pistes d'exploration** : à chaque cycle, les 3 directions les plus prometteuses (terrain jamais exploré, éléments aperçus et pas encore examinés), avec relèvement et distance. `SUIVANT` emmène vers la meilleure ; le pilote IA peut aussi choisir une autre piste (`ALLER`) ou aller voir ce qui l'intrigue à l'image (`ROULER`).
- **Curiosité** : un élément jamais vu est marqué NOUVEAU, examiné et décrit dans une note. La mer est évitée, une piste inaccessible est abandonnée.
- **Retour** : quand la batterie devient juste suffisante pour revenir, l'observation indique RETOUR NÉCESSAIRE et `SUIVANT` ramène le robot au départ.

Résultats : carte (terrain vu en vert, déjà connu en gris, trace, éléments examinés), KML, photos, récit des découvertes par le pilote IA, et section **« Progrès »** : hectares vus, dont jamais explorés auparavant, découvertes, comparés aux missions précédentes lancées du même départ.

10.6 SATER au sol : balise ELT à l'antenne Yagi

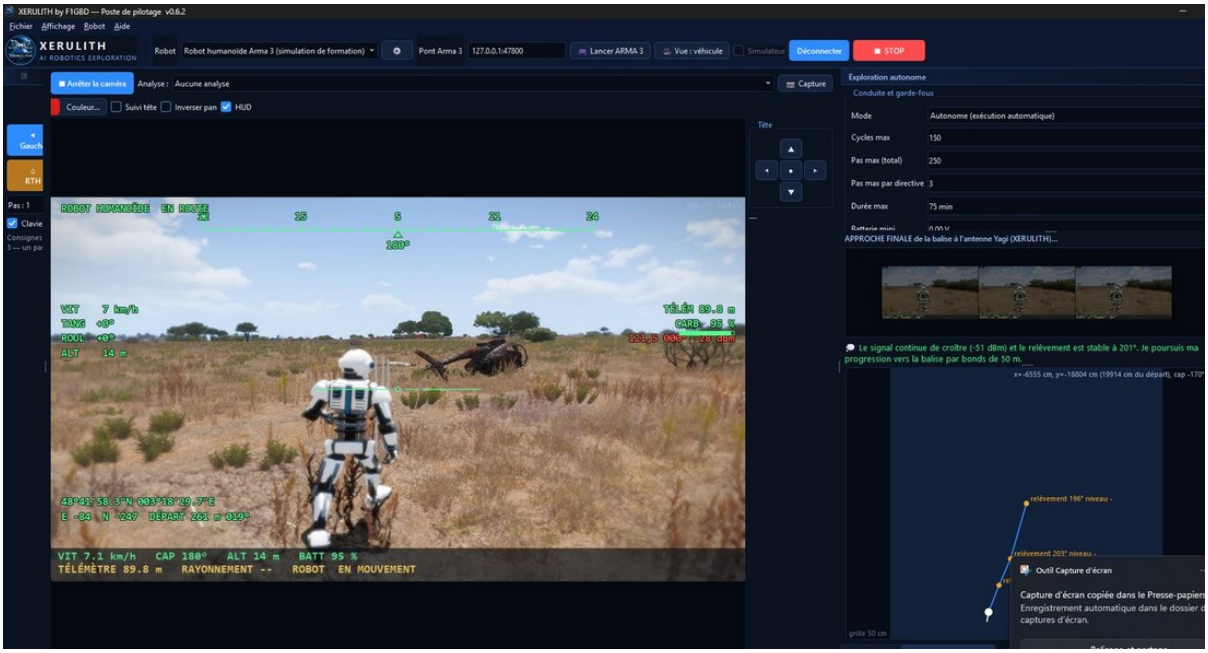
Mission **« Robot humanoïde : SATER, balise ELT 121,5 MHz à l'antenne Yagi »** (120 cycles, 200 pas, 60 min) : une balise de détresse est posée au hasard dans un rayon de 800 m (épave d'un hélicoptère léger et son occupant). Le robot la retrouve à pied, **antenne Yagi 3 éléments** tenue devant lui, comme une équipe au sol.

Le récepteur donne le **niveau reçu (dBm)** dans la direction où pointe l'antenne, **sans relèvement** : c'est en tournant que l'on trouve la direction. Le niveau dépend de la distance, du relief (signal plus faible hors de vue), des bâtiments, des réflexions et du diagramme de la Yagi (maximum dans l'axe, faible sur les côtés et à l'arrière) ; près de la balise, le récepteur sature.

1. `YAGI` : **tour d'horizon radio**, le robot tourne sur lui-même avec une mesure tous les 30°. XERULITH affine le maximum, donne le relèvement, le niveau, la netteté de la mesure (franche, moyenne, brouillée) et propose le pas : `ROULER: <relèvement> <200, 100 ou 50 m>`.
2. Marche au relèvement, puis nouveau tour. XERULITH **recoupe les relèvements** pris à des endroits différents (position estimée de la balise) et signale un endroit déjà visité. Garde-fous : pas de retour au départ avant la balise (sauf batterie faible), et un `ROULER` à contresens d'un relèvement net mesuré sur place est remis au bon cap.
3. Au-dessus de −45 dBm (moins de 150 m environ), **approche finale par XERULITH** : tours rapides, pas de plus en plus courts, repérage de l'épave, examen de l'épave et de l'occupant (sans contact), photo, position notée, puis **balise arrêtée**.

Le son de la balise : comme un vrai récepteur AM, le haut-parleur du poste fait entendre le **souffle** dès que l'antenne est en service ; le son de la balise en sort d'autant plus **net** que le signal est fort. En tournant, on l'entend monter face à la balise et se noyer sur les côtés et à l'arrière. Près de la balise, un **atténuateur automatique** (pas de 10 dB) garde la direction audible jusqu'au bout. Le niveau reçu s'affiche aussi sur le HUD.

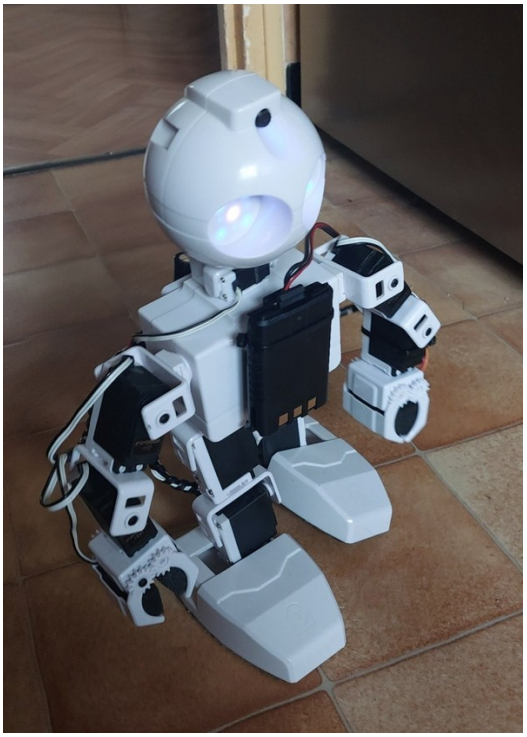
Évaluation instructeur : pour chaque tour d'horizon, relèvement trouvé, relèvement vrai et erreur ; erreur moyenne, plus courte distance atteinte, écart entre la position notée et la balise ; coordonnées dans `balise.csv`, photos de l'épave et de l'occupant.



Arrivée à l'épave : récepteur 121,5 MHz saturé près de la balise, approche finale menée par XERULITH.

11. Monde réel : exploration et découverte libre

Le module d'exploration et de découverte du robot humanoïde d'Arma 3 est porté sur les **robots réels qui marchent au pas**, d'abord l'**EZ-Robot JD Humanoid** (35 cm). Mission **« Exploration et découverte libre (robot réel) »** (catégorie *Découverte*) : la directive première du robot est de **découvrir des choses nouvelles et d'aller vers elles**, sans parcours imposé, prudemment.



Le JD Humanoid piloté par XERULITH, en exploration dans le monde réel.

11.1 Préparer le terrain

- **Sol plat et dégagé**, à l'extérieur ou à l'intérieur ; **jamais sur une table** : le robot ne voit pas le vide. Opérateur à proximité, prêt à faire STOP.
- **Repères ArUco** : imprimez la planche `XERULITH_Reperes_ArUco_10cm.pdf` à l'**échelle 100 %** (vérifiez à la règle : 10 cm de côté) et posez quelques marqueurs debout, bien visibles de la caméra du robot. Ce sont des **repères fixes** : ne les déplacez pas d'une mission à l'autre.
- **Point de départ** : le robot part toujours **du même point, dans la même direction** (par exemple le marqueur n° 0 droit devant) ; c'est le repère commun des missions successives dans ce lieu.
- **Ultrason facultatif** : s'il est déclaré dans le profil (capteur de distance analogique ou I2C sur l'EZ-B), XERULITH l'utilise automatiquement ; sinon, la caméra seule protège le robot.

11.2 Étalonner les pas

La position du robot est **estimée** en comptant ses pas et ses rotations. Mesurez-les une fois sur le sol de l'essai, dans **Étalonnage de l'odométrie** du panneau Exploration :

- **Un pas « Forward »** : faites faire 10 pas au robot, mesurez la distance parcourue et divisez par 10 (3 cm par défaut).
- **Un cycle « Left/Right »** : faites 6 rotations et divisez l'angle total par 6 (15° par défaut).

Un étalonnage juste limite la dérive ; les **repères ArUco** recalent ensuite la position chaque fois qu'ils sont en vue.

11.3 Lancer la mission

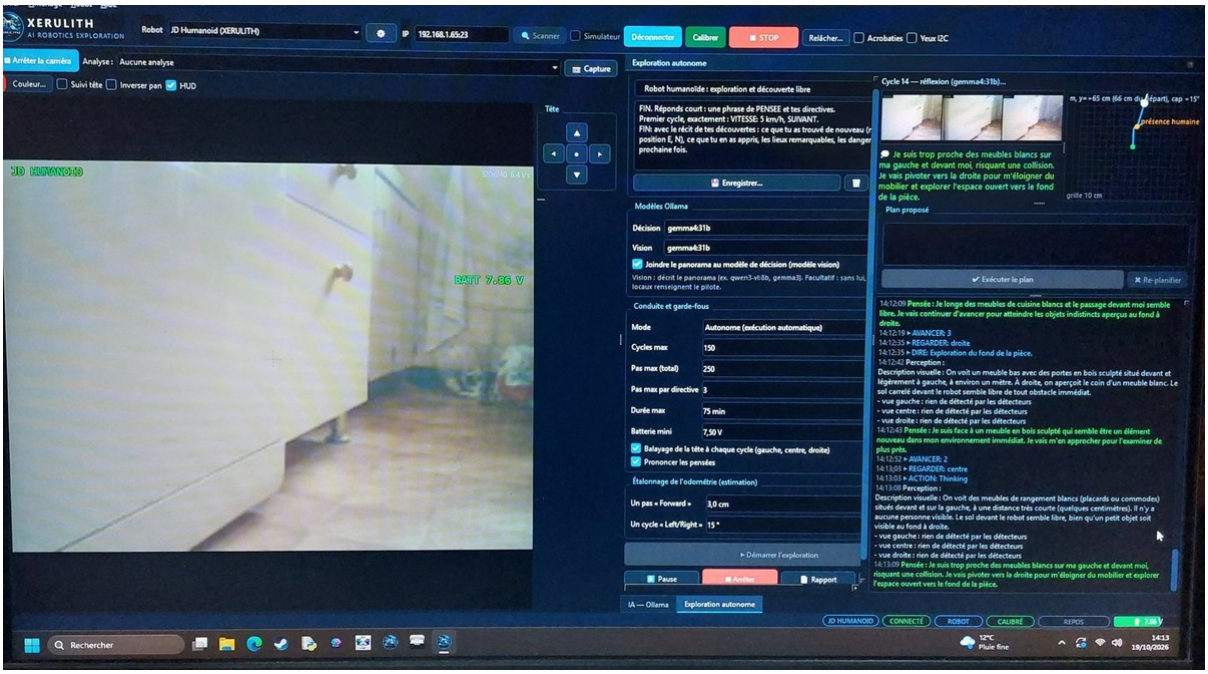
1. Profil **JD Humanoid (XERULITH)** (ou le profil de votre robot), **Connecter**, **Calibrer**, **Démarrer la caméra**.
2. Modèles : décision `gemma3:12b` et vision `qwen3-vl:8b` (ou plus gros, selon le serveur Ollama).
3. Choisissez la mission **« Exploration et découverte libre (robot réel) »**. Remplacez `Lieu : terrain d'essai` par le nom du lieu (jardin, salle…) : chaque lieu a **sa propre mémoire**.
4. Première mission en mode **Confirmer chaque plan** et courte (une vingtaine de cycles), puis en **Autonome**.

> Choisissez bien la mission **« … (robot réel) »** : la mission « Robot humanoïde : exploration et découverte libre » est celle d'Arma 3 et suppose un robot qui marche des dizaines de mètres.

11.4 Ce que fait XERULITH

- **Perception par la caméra** : détecteur d'objets (positions estimées), marqueurs ArUco (distance estimée par leur taille à l'image, recalage de la position), description par le modèle vision, et l'ultrason s'il est monté.
- **Carte de ce qui a été vu** en mailles de 30 cm, gardée dans la mémoire du lieu (`memoire_terrain\reel_<lieu>.json`).
- **Pistes d'exploration** : objet nouveau à examiner, ou terrain jamais vu, avec l'angle dont tourner et la distance. `SUIVANT` y emmène ; `OBSERVER: n` approche un objet détecté, centre la caméra dessus et l'examine.
- **Marche prudente** par tronçons de 2 pas. Avant chaque tronçon, arrêt de sécurité si l'ultrason mesure moins que la distance d'arrêt du profil (20 cm au moins), si un objet occupe le bas et le centre de l'image, ou si l'image ne change plus depuis 2 tronçons (robot bloqué ou tombé). Ce qui est derrière un obstacle n'est plus proposé ; personnes et animaux sont observés de loin ; la batterie est surveillée.

> Sans ultrason, un mur ou un obstacle que le détecteur ne reconnaît pas n'est repéré qu'une fois le robot bloqué. Restez à proximité, STOP (Échap) à portée de main.



Le poste pendant l'exploration du JD : caméra de tête, tour d'horizon, trace estimée et raisonnement du pilote IA (local).

11.5 Rapport et mémoire du lieu

Le rapport de mission contient la **carte** (terrain vu, trajet estimé, repères ArUco, objets), les **photos**, ce qui a été appris, les **leçons** du pilote IA et, à partir de la deuxième mission dans le même lieu, la section **« Progrès »** : ce qui a été vu de nouveau et découvert, comparé aux explorations précédentes.

12. Scripts Python

L'onglet **Script Python** exécute un programme Python complet, sans restriction, avec un objet `robot`. **STOP** interrompt le script au prochain appel de `robot` et arrête le robot. `print` écrit dans la console du panneau. Les scripts s'enregistrent dans `xerulith_data\scripts`.

```
robot.calibrer()
robot.dire("Bonjour, je suis Xérulith.")
robot.action("Wave")
for pan in (-40, 40, 0):
    robot.tete(pan=pan)
    robot.attendre(0.6)
    if robot.voir("visages")["items"]:
        robot.dire("Je vous vois !")
        break
print("Batterie : %.2f V" % robot.batterie())
```

Appel	Effet
`calibrer()`	Tous les servos à 90° (obligatoire en premier)
`action(nom, boucles=1, vitesse=1.0)`	Mouvement de la bibliothèque
`pose(nom)`	Pose de la bibliothèque (STAND…)
`avancer(n)` `reculer(n)` `gauche(n)` `droite(n)`	Déplacements
`servo(port, degres, vitesse=3)`	Commande directe d'un servo
`tete(pan=0, tilt=0)`	Déplacement relatif de la tête
`pince(cote, ouverture)`	« gauche », « droite » ou « les deux », 0 à 100
`yeux(animation)` `yeux_couleur(r, g, b)`	Yeux RGB
`dire(texte)`	Synthèse vocale (attend la fin de la phrase)
`attendre(s)`	Pause interruptible par STOP
`batterie()` `adc(port)` `numerique(port)`	Capteurs
`image()` `voir(mode)`	Dernière image JPEG ; analyse : couleur, qr, visages, aruco, mouvement, objets
`position(port)` `arret()`	Dernière position connue d'un servo ; STOP
`monter(n)` `descendre(n)`	Sous-marin
`regarder(pan, tilt)`	Tête en degrés par rapport au centre
`capteurs()` `distance()`	Mesures des capteurs du profil ; plus petite distance mesurée en cm
`armer()` `desarmer()` `profil()`	Véhicule ArduPilot ; profil du robot (dictionnaire)

13. Configurer Ollama

XERULITH interroge les serveurs Ollama dans l'ordre de la liste de l'onglet IA. Sur le serveur (M1A ou le PC) :

```
ollama pull gemma3:12b
ollama pull qwen3-vl:8b
ollama list
setx OLLAMA_MAX_LOADED_MODELS 2      (puis redémarrer Ollama)
```

- **OLLAMA_MAX_LOADED_MODELS=2** garde les deux modèles chargés ensemble, au lieu de les échanger à chaque cycle.
- XERULITH demande à Ollama de garder les modèles en mémoire 15 minutes entre deux appels, et désactive le raisonnement caché des modèles « thinking ».
- Un serveur distant doit écouter sur le réseau : variable `OLLAMA_HOST=0.0.0.0` sur le serveur, et port 11434 autorisé dans son pare-feu.
- Les modèles d'embedding (bge, nomic-embed…) ne savent pas dialoguer : XERULITH les écarte.

13.1 Ollama Cloud (clé API)

Sans serveur local, XERULITH peut utiliser les modèles hébergés par Ollama sur ollama.com, avec la même interface :

1. Créez une clé sur **ollama.com** → **Settings** → **Keys** (le bouton **?** de l'onglet IA ouvre cette page).
2. Dans l'onglet **IA — Ollama**, choisissez la connexion **Cloud Ollama** ou **Automatique**, puis collez la clé dans **Clé API**. À défaut, la variable d'environnement `OLLAMA_API_KEY` est utilisée.
3. Cliquez **Connexion Ollama** : le voyant affiche « ☁ ollama.com (cloud) » et la liste des modèles cloud se remplit.

La clé n'est envoyée qu'à ollama.com, jamais aux serveurs du réseau local. Cochez **Mémoriser** pour la garder d'une session à l'autre : elle est alors chiffrée par Windows (DPAPI), lisible seulement par votre compte sur ce PC.

Le catalogue cloud diffère du catalogue local : en mode cloud, l'exploration choisit **gemma4:31b** pour la décision et la vision, si ce modèle est proposé. Les quotas d'utilisation dépendent de votre abonnement Ollama ; un quota atteint est signalé en clair.

> Le cloud ajoute de la latence à chaque cycle et envoie les images de la caméra hors du réseau local. Pour une exploration hors connexion Internet, restez en **réseau local**.

14. Dépannage

Symptôme	Cause et solution
« Pas de réponse » ou « connexion refusée » à la connexion	Robot éteint, PC sur un autre réseau, ou **ARC ouvert** (un seul client à la fois). Utilisez **Scanner**.
« Pose inconnue : cliquez d'abord Calibrer »	Après chaque connexion ou relâchement, Calibrer est obligatoire.
« Batterie trop faible pour marcher »	Tension sous 7,45 V : seuls les gestes doux restent permis. Rechargez la batterie.
Un panneau a disparu	Menu **Affichage** : recocher le panneau, ou **Disposition par défaut**.
« Ollama injoignable »	Vérifiez l'adresse du serveur, `OLLAMA_HOST=0.0.0.0` et le pare-feu du serveur.
« Clé API Ollama Cloud refusée »	Clé absente, erronée ou révoquée : recréez-la sur ollama.com → Settings → Keys.
« model … not found »	Modèle absent du serveur : `ollama pull <modèle>`.
Le pilote ne propose aucune directive valide	Modèle trop petit, ou modèle d'embedding choisi : utilisez gemma3:12b.
La tête part du mauvais côté	Cochez **Inverser pan** (panneau Caméra) ; le balayage d'exploration suit ce réglage.
Visages non détectés	OpenCV 5 n'a plus les cascades de Haar : l'exe livré utilise OpenCV 4.
« Aucun battement (HEARTBEAT) reçu »	Véhicule éteint, mauvaise chaîne de connexion, ou port déjà pris par une autre station sol (Mission Planner, QGroundControl).
« Véhicule non armé »	Cliquez **Armer** ; si le véhicule refuse, voir ses contrôles pré-armement dans la station sol.
« Le module pymavlink n'est pas installé »	Installation incomplète : réinstallez XERULITH avec le programme d'installation officiel (`XERULITH-0.4.0-setup.exe`).
« GalaxyRVR injoignable »	PC hors du Wi-Fi GalaxyRVR, application SunFounder encore connectée, ou interrupteur de la carte en position de téléversement.
Capteur « — » dans l'onglet Capteurs	Source absente pour ce contrôleur, clé de carte maison différente, ou pas encore de mesure.
Problème avec Arma 3 (pont, vidéo, drone figé)	Voir le dépannage Arma 3, § 9.13.
L'application se ferme seule	Voir `xerulith_data\logs\console.log` et `crash.log` ; une boîte de message affiche aussi l'erreur.

15. Données et fichiers

Tout ce que XERULITH enregistre est dans le dossier `xerulith_data`, à côté de `XERULITH.exe`. Le menu **Fichier → Ouvrir le dossier de données** y mène directement.

Fichier ou dossier	Contenu
`xerulith.ini`	Réglages et disposition de la fenêtre
`missions.json`	Bibliothèque de missions d'exploration
`profiles\`	Profils de robot (un fichier JSON par profil)
`commandes_vocales.json`	Phrases reconnues et commandes associées
`explorations\`	Une mission par dossier : journal, panoramas, rapport, carte, photos
`memoire_terrain\`	Mémoire de terrain : un fichier par carte d'Arma 3 (`Altis.json`…) ou par lieu réel (`reel_<lieu>.json`) ; le supprimer efface ce qui a été appris
`scripts\`	Scripts Python de l'opérateur
`captures\`	Images de la caméra
`logs\`	Journal du jour, console.log, crash.log

16. Crédits

- Protocole EZ-B v4, bibliothèque de mouvements du JD, détecteurs de vision et simulateur : projet **Commander** de Dustin Hightower (github.com/drhalto/commander), licence MIT.
- EZ-Robot, ARC et JD-Humanoid sont des marques d'EZ-Robot Inc. XERULITH n'est pas affilié à EZ-Robot.
- Modèles d'IA servis par Ollama : Gemma 3 (Google), Qwen3-VL (Alibaba).
- MAVLink : **pymavlink** (projet ArduPilot), licence LGPL-3.0 ; liaison série : **pyserial**, licence BSD ; WebSocket : **websockets**, licence BSD.
- GalaxyRVR : protocole des projets galaxy-rvr et ai-camera-firmware de SunFounder. SunFounder et GalaxyRVR sont des marques de SunFounder ; XERULITH n'est pas affilié à SunFounder.
- Simulation de vol : mod **Pythia** (github.com/overfl0/Pythia). Arma 3 est une marque de Bohemia Interactive ; XERULITH n'est pas affilié à Bohemia Interactive.

XERULITH — by F1GBD (c) 2026